

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-acfee8e480ea63f32.jsonl`  
- SHA-256: `6846b9edddf82d3dd1fc489b599aa066406a065d2fd3113a06707821ff138991`  
- Source modified: `2026-06-10T16:21:13+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:13:27.752Z)

You are doing a SECURITY-POSTURE audit of muneem (C:/Users/avidu/Projects/muneem), a single-user local-first tool that aggregates the user's ENTIRE financial life into one encrypted SQLite ledger on a Windows machine. Threat model: the repo is private but pushed to GitHub; the testdata dir contains real bank statements; secrets live in the OS keychain; nothing sensitive may ever reach git history, GitHub, logs, or any cloud service.

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all --<path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.

Audit these dimensions, reading whatever code/config you need (src/muneem fully available; .gitignore; .gitattributes; .github/hooks/pre-commit, .github/hooks/pre-push, .github/hooks/precommit_check.py; .github/workflows/ci.yml; Makefile; pyproject.toml; tests/test_repo_hygiene.py, tests/test_redaction.py, tests/test_secrets.py, tests/test_db_invariants.py):

1. Repo hygiene GROUND TRUTH: run git ls-files to enumerate everything tracked; verify NONE of testdata/credentials/, testdata/*-statements/, testdata/snapshots/, password rules, .env, or DB files are tracked. Then check HISTORY: git log --all --diff-filter=A --name-only -- 'testdata/credentials' 'testdata/*statements*' 'testdata/snapshots' '*.env' '*client_secret*' ? has anything sensitive EVER been committed? Also: is testdata/promo-emails/expected.yaml clean? Are the snapshots JSONs derived from REAL statements sitting unencrypted on disk in the repo dir (information-at-rest risk even if untracked)?
2. The pre-commit guard: read .github/hooks/precommit_check.py ? what patterns does it block? What sensitive file types slip through (e.g. snapshots/*.json, *.yaml password rules, recovery.json, *.db in other dirs)? Is it actually installed by default (hooks need 'make hooks')? Does CI enforce equivalent checks (test_repo_hygiene.py) ? and does that test run on every push?
3. Secrets handling: keyring usage (service/account naming collisions, fallback when keyring unavailable ? does anything fall back to plaintext file?), OAuth token storage location and permissions, statement passwords cached in keychain, password_rules.yaml location (is it in the repo dir or user config dir? could it be committed?), recovery.json default write location.
4. Sensitive data in process artifacts: statement passwords on argv or env (visible in process lists)? Raw PDF text in temp files (extract.py/ocr.py temp handling, cleanup on crash)? Sensitive values in exception messages/tracebacks that reach logs or CI output? The RedactingFilter as backstop ? grep all logger/print calls in src/ for raw row/amount/account leakage.

5. DB-at-rest guarantees: can any production code path create or leave a PLAINTEXT ledger (db.connect vs connect_encrypted; the :memory:/test escape hatch reachable in prod?); backup files keyed how; recovery file brute-force-ability (Argon2id params).
6. CI: does ci.yml ever upload artifacts/logs that could contain extracted text? Are real-statement tests genuinely impossible on GitHub runners? Any secrets in workflow?
7. Supply chain: pyproject dependency pins, the license-allowlist test, any dependency with native binaries fetched at install (sqlcipher3 wheels), pip hash-checking absence.

Report findings with exact evidence (paths, hook patterns, code lines ? never sensitive values). Severity honestly. Also note what is done WELL (the strengths list matters for the final report). Structured output only.

2. user (2026-06-10T16:13:34.901Z)

```
.gitattributes
.githubhooks/pre-commit
.githubhooks/pre-push
.githubhooks/precommit_check.py
.github/workflows/ci.yml
.gitignore
CLAUDE.md
DESIGN.md
Makefile
README.md
docs/CURRENT_STATE.md
docs/DEPENDENCIES.md
docs/ROADMAP.md
docs/TESTING.md
docs/encryption-at-rest.md
docs/feedbackJune2026.md
docs/ingesting-statements.md
docs/onboarding-a-bank.md
docs/parser-architecture.md
docs/prior-art-email-mining.md
docs/research-open-banking-india.md
docs/schema.md
pyproject.toml
src/muneem/__init__.py
src/muneem/cli.py
src/muneem/config.py
src/muneem/db.py
src/muneem/db_key.py
src/muneem/errors.py
src/muneem/extract.py
src/muneem/ingestion/__init__.py
src/muneem/ingestion/base.py
src/muneem/ingestion/gmail.py
src/muneem/ingestion/imap.py
src/muneem/ingestion/outlook.py
src/muneem/log.py
src/muneem/migrations.py
src/muneem/ocr.py
src/muneem/offers.py
src/muneem/parsers/__init__.py
src/muneem/parsers/au.py
src/muneem/parsers/axis.py
src/muneem/parsers/base.py
src/muneem/parsers/clustering.py
src/muneem/parsers/concepts.py
src/muneem/parsers/engine.py
src/muneem/parsers/hdfc.py
src/muneem/parsers/profiles.py
src/muneem/parsers/savings.py
src/muneem/parsers/validation.py
src/muneem/password_rules.py
```

src/muneem/recovery.py
src/muneem/redaction.py
src/muneem/secrets.py
src/muneem/statement_unlock.py
src/muneem/unlock.py
testdata/promo-emails/README.md
testdata/promo-emails/airasia-1.pdf
testdata/promo-emails/airasia-2.pdf
testdata/promo-emails/airasia-3.pdf
testdata/promo-emails/dataquest.pdf
testdata/promo-emails/expected.yaml
testdata/promo-emails/flipkart.pdf
testdata/promo-emails/makemytrip-1.pdf
testdata/promo-emails/makemytrip-2.pdf
testdata/promo-emails/swiggy.pdf
testdata/promo-emails/tataneu.pdf
testdata/promo-emails/unacademy.pdf
tests/conftest.py
tests/ingestion/test_gmail.py
tests/ingestion/test_providers.py
tests/parsers/test_au.py
tests/parsers/test_axis.py
tests/parsers/test_clustering.py
tests/parsers/test_engine.py
tests/parsers/test_hdfc.py
tests/parsers/test_local_au.py
tests/parsers/test_local_axis.py
tests/parsers/test_parse_errors.py
tests/parsers/test_registry.py
tests/parsers/test_savings.py
tests/parsers/test_validation.py
tests/synthpdf.py
tests/test_cli.py
tests/test_db.py
tests/test_db_crypto.py
tests/test_db_crypto_ops.py
tests/test_db_invariants.py
tests/test_dependency_licenses.py
tests/test_errors.py
tests/test_eval_cli.py
tests/test_extract.py
tests/test_log.py
tests/test_migrations.py
tests/test_ocr.py
tests/test_offers.py
tests/test_password_rules.py
tests/test_precommit_hook.py
tests/test_promo_regression.py
tests/test_provenance.py
tests/test_recovery.py
tests/test_redaction.py
tests/test_regression_synth.py
tests/test_repo_hygiene.py
tests/test_secrets.py
tests/test_smoke.py
tests/test_statement_unlock.py
tests/test_unlock.py
tests/test_verifier_properties.py
tools/gmail_auth.py
tools/imap_auth.py
tools/outlook_auth.py

3. user (2026-06-10T16:13:36.138Z)

COMMIT 9f9de66 Harden secrets / password handling + attribute external calls (#37)

tests/test_password_rules.py

COMMIT bf87d9d Harden secrets / password handling + attribute external calls
tests/test_password_rules.py

COMMIT d2397d0 Add Axis Credit Card Parser and rigorous local test suite (#10)
src/muneem/password_rules.py

4. user (2026-06-10T16:13:45.015Z)

```
1      #!/usr/bin/env python3
2      """muneem sensitive-file guard (invoked by the pre-commit hook).
3
4      Refuses to *stage* files that policy says must never be committed: real
5      statements (any PDF, or anything under ``testdata/`` outside the allowed
6      corpora), OAuth secrets, password rules, local databases, key material, and
7      ``.env`` files. This is the PRE-commit backstop to ``.gitignore`` ? it stops the
8      ``git add -f`` / ``git add -A`` mistake before it becomes a commit (the very
9      thing that caused an earlier statement leak). The post-commit complement is
10     ``tests/test_repo_hygiene.py``.
11
12     Exit 0 when nothing sensitive is staged, 1 otherwise. A deliberate, reviewed
13     exception can be committed with ``git commit --no-verify``.
14     """
15
16     from __future__ import annotations
17
18     import fnmatch
19     import os
20     import subprocess
21     import sys
22
23     # Only these subtrees of testdata/ are non-sensitive and may be committed.
24     ALLOWED_PREFIXES = ("testdata/promo-emails/", "testdata/synthetic/")
25     SECRET_NAMES = {"credentials.json", "token.json", "token.pickle", "passwords.txt"}
26
27
28     def is_sensitive(path: str) -> str | None:
29         """Return a human-readable reason if ``path`` must not be committed, else None."""
30         p = path.replace("\\", "/").strip()
31         if not p or p.startswith(ALLOWED_PREFIXES):
32             return None
33         name = p.rsplit("/", 1)[-1]
34         low = p.lower()
35         nlow = name.lower()
36         if p.startswith("testdata/"):
37             return "under testdata/ ? sensitive by default (only promo-emails/ and
38             synthetic/ allowed)"
39         if low.endswith(".pdf"):
40             return "PDF outside the allowed corpora (real statements must stay local)"
41         if low.endswith((".db", ".sqlite", ".sqlite3")):
42             return "local database (may contain parsed statement data)"
43         if low.endswith(".plaintext.bak"):
44             return "plaintext database backup (cleartext from 'muneem db encrypt'
45             migration)"
46         if fnmatch.fnmatch(nlow, "*recovery*.json"):
47             return "DB-key recovery file (passphrase-wrapped key ? keep it off the repo)"
48         if low.endswith((".pem", ".key")):
49             return "key material"
50         if nlow in SECRET_NAMES:
51             return "OAuth / credential secret"
52         if nlow.startswith("oauth_token"):
53             return "OAuth token"
54         # Block the rules *data* file (password_rules.yaml/.yml/.json/?) but not the loader
55         module
```

```

53     # (src/muneem/password_rules.py is code, not a secret).
54     if fnmatch.fnmatch(nlow, "password_rules.*") and not nlow.endswith(".py"):
55         return "password rules"
56     if (nlow == ".env" or nlow.startswith(".env.")) and nlow != ".env.example":
57         return ".env file (may contain secrets)"
58     return None
59
60
61 def _staged_paths() -> list[str]:
62     # Tests inject a newline-separated list via PRECOMMIT_TEST_FILES (no git needed).
63     override = os.environ.get("PRECOMMIT_TEST_FILES")
64     if override is not None:
65         return [ln for ln in override.replace("\r\n", "\n").split("\n") if ln.strip()]
66     # -z keeps paths with spaces/unicode intact (the real statements have spaces).
67     out = subprocess.run(
68         ["git", "diff", "--cached", "--name-only", "-z", "--diff-filter=ACM"],
69         capture_output=True,
70         text=True,
71         check=True,
72     ).stdout
73     return [p for p in out.split("\0") if p.strip()]
74
75
76 def main() -> int:
77     offenders = [(p, reason) for p in _staged_paths() if (reason := is_sensitive(p))]
78     if not offenders:
79         return 0
80     print("muneem pre-commit: refusing to commit sensitive file(s):", file=sys.stderr)
81     for p, reason in offenders:
82         print(f"    x {p}\n        {reason}", file=sys.stderr)
83     print(
84         "\nThese must never reach git (CLAUDE.md security rules; see .gitignore).\n"
85         "If this is a deliberate, reviewed exception: git commit --no-verify",
86         file=sys.stderr,
87     )
88     return 1
89
90
91 if __name__ == "__main__":
92     raise SystemExit(main())
93

```

5. user (2026-06-10T16:13:45.121Z)

```

1     # --- Sensitive data ? NEVER commit ---
2     # Real financial documents stay local only. Everything under testdata/ is sensitive by
3     # default ? ONLY the non-sensitive promo corpus and committed synthetic fixtures are
4     # Real statements in ANY subdir (e.g. testdata/hdfc-statements/) must never be
5     testdata/*
6     !testdata/promo-emails/
7     !testdata/synthetic/
8     *.db
9     *.sqlite
10    *.sqlite3
11    muneem.db*
12
13    # Secrets and credentials
14    .env
15    .env.*
16    !.env.example
17    *.pem
18    *.key
19    credentials.json

```

```
20 token.json
21 token.pickle
22 oauth_token*
23 password_rules.yaml
24 password_rules.yml
25 password_rules.json
26 passwords.txt
27
28 # Logs (may contain extracted statement text ? sanitize before committing any log)
29 *.log
30 logs/
31
32 # --- Python ---
33 __pycache__/
34 *.py[cod]
35 *$py.class
36 *.so
37 .Python
38 build/
39 develop-eggs/
40 dist/
41 downloads/
42 eggs/
43 .eggs/
44 lib/
45 lib64/
46 parts/
47 sdist/
48 var/
49 wheels/
50 *.egg-info/
51 .installed.cfg
52 *.egg
53 MANIFEST
54 .venv/
55 venv/
56 env/
57 ENV/
58 .pytest_cache/
59 .mypy_cache/
60 .ruff_cache/
61 .coverage
62 htmlcov/
63 .tox/
64
65 # --- Go ---
66 *.exe
67 *.exe~
68 *.dll
69 *.dylib
70 *.test
71 *.out
72 vendor/
73
74 # --- Node (if a UI later lands) ---
75 node_modules/
76 npm-debug.log*
77 yarn-debug.log*
78 yarn-error.log*
79 .next/
80 .nuxt/
81 dist/
82
83 # --- IDE / OS ---
84 .vscode/
```

```

85     .idea/
86     *.swp
87     *.swo
88     *~
89     .DS_Store
90     Thumbs.db
91     desktop.ini
92
93     # --- Jupyter (in case notebooks are used for exploration) ---
94     .ipynb_checkpoints/
95
96     # --- Scratch / local-only ---
97     scratch/
98     local/
99     tmp/
100    ruff-bin/
101    ruff.zip
102
103

```

6. user (2026-06-10T16:13:45.484Z)

```

1     #!/bin/sh
2     # muneem pre-commit guard ? blocks staging of sensitive files (real statements,
3     # secrets, local DBs). See .githooks/precommit_check.py for the rules.
4     # Locates a Python interpreter so this works on Linux/WSL and Git-for-Windows.
5     # Enable with: git config core.hooksPath .githooks (or `make hooks`).
6     dir=$(dirname "$0")
7     for py in python3 python py; do
8         if command -v "$py" >/dev/null 2>&1; then
9             exec "$py" "$dir/precommit_check.py"
10        fi
11    done
12    echo "muneem pre-commit: no python interpreter found; skipping the sensitive-file
guard." >&2
13    echo " (install python to enable; tests/test_repo_hygiene.py is the post-commit
backstop)" >&2
14    exit 0
15

```

7. user (2026-06-10T16:13:45.849Z)

```

1     #!/bin/sh
2     # muneem pre-push gate ? runs lint + the offline test suite (including the DB invariant
3     # tripwires in tests/test_db_invariants.py) so only green code reaches GitHub. This is
the
4     # LOCAL substitute for server-side branch protection, which GitHub gates behind Pro /
public
5     # repos (this repo is private by design).
6     #
7     # Bypass intentionally (the "explicitly allowed" escape hatch) with:
8     #     git push --no-verify
9     #
10    # Enable with: git config core.hooksPath .githooks (or `make hooks`).
11
12    root=$(git rev-parse --show-toplevel 2>/dev/null) || exit 0
13
14    # Prefer the project venv (it has ruff + pytest); fall back to a PATH python.
15    if [ -x "$root/.venv/bin/python" ]; then
16        py="$root/.venv/bin/python"
17    elif [ -x "$root/.venv/Scripts/python.exe" ]; then
18        py="$root/.venv/Scripts/python.exe"
19    else
20        py=""
21    for cand in python3 python py; do

```

```

22         if command -v "$scand" >/dev/null 2>&1; then py="$scand"; break; fi
23     done
24 fi
25
26     if [ -z "$py" ]; then
27         echo "muneem pre-push: no python/venv found; skipping the lint+test gate (CI still
runs)." >&2
28         exit 0
29     fi
30
31     # If the dev tooling isn't installed in this interpreter, skip rather than false-block a
push.
32     if ! "$py" -m pytest --version >/dev/null 2>&1; then
33         echo "muneem pre-push: pytest not available in $py; skipping gate (run 'make
install')." >&2
34         echo " CI still runs on push; or override now with: git push --no-verify" >&2
35         exit 0
36     fi
37
38     fail() {
39         echo "muneem pre-push: $1 ? push BLOCKED." >&2
40         echo " Fix it, or bypass intentionally with: git push --no-verify" >&2
41         exit 1
42     }
43
44     echo "muneem pre-push: ruff check ..." >&2
45     "$py" -m ruff check "$root" || fail "ruff check failed"
46     echo "muneem pre-push: ruff format --check ..." >&2
47     "$py" -m ruff format --check "$root" || fail "ruff format check failed"
48     if "$py" -m mypy --version >/dev/null 2>&1; then
49         echo "muneem pre-push: mypy ..." >&2
50         (cd "$root" && "$py" -m mypy) || fail "mypy failed"
51     fi
52     echo "muneem pre-push: pytest (offline suite) ..." >&2
53     "$py" -m pytest -q "$root" || fail "tests failed"
54
55     echo "muneem pre-push: all checks passed." >&2
56     exit 0
57

```

8. user (2026-06-10T16:13:54.440Z)

(Bash completed with no output)

9. user (2026-06-10T16:13:55.411Z)

```

testdata/promo-emails/airasia-1.pdf
testdata/promo-emails/airasia-2.pdf
testdata/promo-emails/airasia-3.pdf
testdata/promo-emails/dataquest.pdf
testdata/promo-emails/flipkart.pdf
testdata/promo-emails/makemytrip-1.pdf
testdata/promo-emails/makemytrip-2.pdf
testdata/promo-emails/swiggy.pdf
testdata/promo-emails/tataneu.pdf
testdata/promo-emails/unacademy.pdf

```

10. user (2026-06-10T16:13:56.425Z)

```

testdata/aubank-statements/
testdata/axis-statements/
testdata/credentials/
testdata/hdfc-statements/
testdata/snapshots/

```

```
---
aubank-statements
axis-statements
credentials
hdfc-statements
promo-emails
snapshots
```

11. user (2026-06-10T16:14:03.272Z)

```
1      """Password rule loader for statement decryption (hybrid strategy support).
2
3      Prefers the user config dir (via platformdirs, e.g. ~/.config/muneem/password_rules.yaml
4      on Linux) for real rules. Falls back to legacy CWD "password_rules.yaml" (common during
5      dev) with a DeprecationWarning. **Never commit real rules or the file itself** ?
6      .gitignore`
7      blocks `password_rules.*`.
8
9      See DESIGN.md §8.2, CLAUDE.md security rules, and docs/ingesting-statements.md for the
10     file
11     format. Entries are either ``<bank>: <password>`` or a top-level ``candidates: [...]``
12     list.
13
14     """
15
16     from __future__ import annotations
17
18     import warnings
19     from pathlib import Path
20     from typing import Any
21
22     import yaml
23
24     from muneem.config import CONFIG_DIR
25     from muneem.errors import ConfigError
26
27     def _load_rules() -> dict[str, Any]:
28         """Load the rules dict from the config dir (preferred) or legacy CWD file; ``{}`` if
29         absent.
30
31         Resolution order:
32         1. ``<config_dir>/password_rules.yaml`` (recommended location)
33         2. ``./password_rules.yaml`` (legacy, emits DeprecationWarning)
34         """
35         for path, is_legacy in (
36             (CONFIG_DIR / "password_rules.yaml", False),
37             (Path("password_rules.yaml"), True),
38         ):
39             if path.exists():
40                 if is_legacy:
41                     warnings.warn(
42                         "Loading password_rules from CWD (./password_rules.yaml) is
43                         deprecated. "
44                         f"Move it to {CONFIG_DIR / 'password_rules.yaml'} for future-
45                         proofing. "
46                         "See docs and CLAUDE.md (never commit real password material).",
47                         DeprecationWarning,
48                         stacklevel=3,
49                     )
50                 with path.open(encoding="utf-8") as f:
51                     try:
52                         data = yaml.safe_load(f)
53                     except yaml.YAMLError as exc:
54                         # Never echo the file's contents in the error ? they are password
55                         material.
```

```

49             raise ConfigError(
50                 f"could not parse password rules ({path.name}): invalid YAML"
51             ) from exc
52         if data is None:
53             return {}
54         if not isinstance(data, dict):
55             raise ConfigError(
56                 f"password rules ({path.name}) must be a mapping of <bank>:
<password>"
57             )
58         return data
59     return {}
60
61
62     def get_password_for_bank(bank_name: str) -> str | None:
63         """Return the password (or derived rule string) for a bank, or ``None``."""
64         val = _load_rules().get(bank_name)
65         return str(val) if val else None
66
67
68     def candidate_passwords() -> list[str]:
69         """Every configured password to try as an unlock candidate, bank-agnostic.
70
71         The ingest path resolves the password before it knows the issuer (it needs the
72         decrypted
73         text to pick a parser), so it tries all configured passwords. Collects each per-bank
74         value
75         plus any entries under a top-level ``candidates`` list; order preserved, de-
76         duplicated.
77         Quote numeric passwords in the YAML so leading zeros survive.
78         """
79         rules = _load_rules()
80         collected: list[str] = []
81         for key, val in rules.items():
82             if key == "candidates" and isinstance(val, list):
83                 collected.extend(str(v) for v in val if v not in (None, ""))
84             elif isinstance(val, str | int | float):
85                 collected.append(str(val))
86         seen: set[str] = set()
87         unique: list[str] = []
88         for pw in collected:
89             if pw and pw not in seen:
90                 seen.add(pw)
91                 unique.append(pw)
92         return unique

```

12. user (2026-06-10T16:14:03.890Z)

```

from pathlib import Path

import yaml

def get_password_for_bank(bank_name: str) -> str | None:
    path = Path("password_rules.yaml")
    if not path.exists():
        return None
    with path.open(encoding="utf-8") as f:
        rules = yaml.safe_load(f)
    if not rules:
        return None
    return str(rules.get(bank_name, "")) or None

```

13. user (2026-06-10T16:14:13.967Z)

```

1      """Secret storage via the OS keychain.
2
3      OAuth tokens, cached statement passwords, and the DB encryption key are stored here
(Windows
4      Credential Manager / macOS Keychain / Secret Service), never in the repo, on disk in
plaintext, or
5      in logs. See DESIGN.md section 5.
6
7      External-call discipline (every keyring call is wrapped with attribution):
8      - **read** (:func:`get_secret`) is *graceful* ? a missing or unavailable backend returns
`None`
9      (absence is a valid state; this also keeps headless CI happy).
10     - **write** (:func:`set_secret`) is *loud* ? a failure raises
:class:`~muneem.errors.KeychainError`,
11     because silently losing a write (e.g. the DB key) would be dangerous. The secret
*name* is
12     included for diagnosis; the value never is.
13     """
14
15     from __future__ import annotations
16
17     import keyring
18     from keyring.errors import KeyringError, PasswordDeleteError
19
20     from muneem.errors import KeychainError
21
22     _SERVICE = "muneem"
23
24
25     def get_secret(name: str) -> str | None:
26         """Return the stored secret, or `None` if absent or the keychain is
unavailable."""
27         try:
28             return keyring.get_password(_SERVICE, name)
29         except KeyringError:
30             return None
31
32
33     def set_secret(name: str, value: str) -> None:
34         """Store (or replace) a secret in the OS keychain. Raises KeychainError on
failure."""
35         try:
36             keyring.set_password(_SERVICE, name, value)
37         except KeyringError as exc:
38             # Name only ? never the value ? in the message (it may be a password or the DB
key).
39             raise KeychainError(f"could not store secret {name!r} in the OS keychain") from
exc
40
41
42     def delete_secret(name: str) -> None:
43         """Delete a secret if present; a no-op if it is not (or the keychain is
unavailable)."""
44         try:
45             keyring.delete_password(_SERVICE, name)
46         except (PasswordDeleteError, KeyringError):
47             pass
48

```

14. user (2026-06-10T16:14:14.269Z)

```

1      """The ledger's encryption key ? minted once, kept in the OS keychain.
2
3      muneem's local ledger is **encrypted at rest** with SQLCipher (DESIGN.md §5.4 rule 4).
4      The 256-bit key is generated on first use and stored in the OS keychain via

```

```

5      :mod:`muneem.secrets` (Windows Credential Manager / macOS Keychain / Secret Service) ?
6      never on disk in plaintext, never in the repo, never logged.
7
8      It is handed to SQLCipher in **raw-key form** (`PRAGMA key = 'x'<64 hex>'`), which
uses
9      the 256-bit key directly and *skips* the passphrase KDF. That is correct here because
the
10     key is already full-entropy random; KDF stretching only buys security for low-entropy
human
11     passphrases (an opt-in we may add later ? see ROADMAP D.2 phased plan).
12     ""
13
14     from __future__ import annotations
15
16     import secrets as _stdlib_secrets
17
18     from muneem import secrets as _keychain
19
20     # Keychain entry name (under the shared "muneem" service) holding the 64-hex-char DB
key.
21     _DB_KEY_NAME = "db-key"
22
23
24     def get_or_create_key() -> str:
25         """Return the ledger's 256-bit key as 64 hex chars, minting+storing one on first
use."""
26         existing = _keychain.get_secret(_DB_KEY_NAME)
27         if existing:
28             return existing
29         key = _stdlib_secrets.token_bytes(32).hex()
30         _keychain.set_secret(_DB_KEY_NAME, key)
31         return key
32
33
34     def peek_key() -> str | None:
35         """Return the stored key, or ``None`` if none has been minted yet. Never creates
one."""
36         return _keychain.get_secret(_DB_KEY_NAME)
37
38
39     def new_key() -> str:
40         """Generate a fresh random 256-bit key as 64 hex chars (not stored)."""
41         return _stdlib_secrets.token_bytes(32).hex()
42
43
44     def set_key(key: str) -> None:
45         """Overwrite the stored DB key ? used by key rotation, **after** the file has been
rekeyed."""
46         _keychain.set_secret(_DB_KEY_NAME, key)
47

```

15. user (2026-06-10T16:14:21.437Z)

```

1      """Local SQLite store: the ``documents`` and ``offers`` slice of the schema.
2
3      This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
4      (accounts, transactions, holdings, merchants, items) grows with later milestones.
5      ``documents.sha256`` is UNIQUE so re-ingesting the same file is a no-op.
6      ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
7      ""
8
9      from __future__ import annotations
10
11     import os
12     import sqlite3

```

```

13     from collections.abc import Iterator
14     from contextlib import contextmanager
15     from pathlib import Path
16
17     from muneem.errors import EncryptedDatabaseError # re-exported so
db.EncryptedDatabaseError works
18
19     SCHEMA_VERSION = 4
20
21     # Unencrypted SQLite files start with this 16-byte magic header; an encrypted
(SQLCipher)
22     # file does not, so we can tell them apart without a key.
23     _SQLITE_MAGIC = b"SQLite format 3\x00"
24
25     _SCHEMA = """
26     CREATE TABLE IF NOT EXISTS documents (
27         id            INTEGER PRIMARY KEY,
28         source         TEXT NOT NULL,
29         external_id    TEXT,
30         sender         TEXT,
31         subject        TEXT,
32         received_date  TEXT,
33         doc_type       TEXT,
34         sha256         TEXT NOT NULL UNIQUE,
35         path           TEXT,
36         status         TEXT NOT NULL DEFAULT 'parsed',
37         created_at     TEXT NOT NULL DEFAULT (datetime('now'))
38     );
39
40     CREATE TABLE IF NOT EXISTS offers (
41         id            INTEGER PRIMARY KEY,
42         document_id   INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
43         merchant      TEXT,
44         promo_code    TEXT,
45         description   TEXT,
46         expiry_raw    TEXT,
47         expiry_type   TEXT,
48         source        TEXT NOT NULL DEFAULT 'text',
49         redeemed      INTEGER NOT NULL DEFAULT 0,
50         source_text   TEXT
51     );
52
53     CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);
54
55     -- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
56     -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
57     -- Normalization into a unified view layer is planned for a later phase
58     -- once all bank schemas are stabilized.
59     CREATE TABLE IF NOT EXISTS axis_transactions (
60         id INTEGER PRIMARY KEY,
61         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
62         account_number TEXT,
63         txn_date TEXT,
64         value_date TEXT,
65         particulars TEXT,
66         debit REAL,
67         credit REAL,
68         balance REAL
69     );
70
71     CREATE TABLE IF NOT EXISTS axis_cc_transactions (
72         id INTEGER PRIMARY KEY,
73         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
74         account_number TEXT,
75         txn_date TEXT,

```

```

76         particulars TEXT,
77         merchant_category TEXT,
78         amount REAL,
79         type TEXT
80     );
81
82     CREATE TABLE IF NOT EXISTS au_transactions (
83         id INTEGER PRIMARY KEY,
84         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
85         account_number TEXT,
86         txn_date TEXT,
87         value_date TEXT,
88         particulars TEXT,
89         debit REAL,
90         credit REAL,
91         balance REAL
92     );
93
94     CREATE TABLE IF NOT EXISTS hdfc_transactions (
95         id INTEGER PRIMARY KEY,
96         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
97         account_number TEXT,
98         txn_date TEXT,
99         value_date TEXT,
100        particulars TEXT,
101        ref TEXT,
102        withdrawal REAL,
103        deposit REAL,
104        closing_balance REAL
105    );
106    """
107
108
109    def _raw_key_pragma(key: str) -> str:
110        """The SQLCipher raw-key PRAGMA: use the 256-bit ``key`` (64 hex chars) directly, no
KDF."""
111        return f"PRAGMA key = \"x'{key}'\""
112
113
114    def is_plaintext_sqlite(db_path: Path | str) -> bool:
115        """True iff the file exists and begins with the *unencrypted* SQLite magic header.
116
117        Lets us refuse to silently treat a plaintext ledger as encrypted (and vice-versa)
118        without needing the key.
119        """
120        try:
121            with open(db_path, "rb") as fh:
122                return fh.read(16) == _SQLITE_MAGIC
123        except OSError:
124            return False
125
126
127    def _harden_permissions(path: str) -> None:
128        """Best-effort: restrict the DB file to its owner (0600). No-op on failure or
``:memory:``."""
129        if path == ":memory:":
130            return
131        try:
132            os.chmod(path, 0o600)
133        except OSError:
134            pass
135
136
137    def _open_encrypted(path: str, key: str) -> sqlite3.Connection:
138        """Open an encrypted connection via SQLCipher, verifying the key before returning.

```

```

139
140     A keyed open NEVER silently downgrades to plaintext: if the binding is missing, or
the
141     key is wrong (or the file is not an encrypted muneem ledger), this raises.
142     """
143     try:
144         import sqlcipher3
145     except ImportError as exc: # pragma: no cover - exercised only without the binding
146         raise EncryptedDatabaseError(
147             "database encryption requested but the 'sqlcipher3' package is not
installed"
148             ) from exc
149
150     conn = sqlcipher3.connect(path)
151     conn.execute(_raw_key_pragma(key)) # MUST be the first statement on the connection
152     try:
153         conn.execute("SELECT count(*) FROM sqlite_master") # forces key verification
154     except sqlcipher3.DatabaseError as exc:
155         conn.close()
156         raise EncryptedDatabaseError(
157             f"could not open encrypted database {Path(path).name}: wrong key, or not an
"
158             "encrypted muneem ledger"
159             ) from exc
160     conn.row_factory = sqlcipher3.Row
161     return conn
162
163
164     @contextmanager
165     def connect(db_path: Path | str, *, key: str | None = None) ->
Iterator[sqlite3.Connection]:
166         """Open a connection (commit on success, always close). Parent dirs created.
167
168         With ``key`` (a 64-hex-char SQLCipher key) the database is **encrypted at rest** via
169         SQLCipher. Without it, a plaintext stdlib :mod:`sqlite3` connection is used ? the
path for
170         ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
171         :func:`_open_encrypted`); the production ledger is opened via
:func:`connect_encrypted`.
172         """
173         path = str(db_path)
174         if path != ":memory:":
175             Path(path).parent.mkdir(parents=True, exist_ok=True)
176         if key is not None:
177             conn = _open_encrypted(path, key)
178             _harden_permissions(path)
179         else:
180             conn = sqlite3.connect(path)
181             conn.row_factory = sqlite3.Row
182             conn.execute("PRAGMA foreign_keys = ON")
183         try:
184             yield conn
185             conn.commit()
186         finally:
187             conn.close()
188
189
190     @contextmanager
191     def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:
192         """Open the real on-disk ledger, encrypted with the keychain-managed key.
193
194         The production path: resolves (or mints, on first use) the 256-bit key from the OS
195         keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
SQLite
196         file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic

```

```

197     "file is not a database" error.
198     """
199     from muneem.db_key import get_or_create_key
200
201     path = str(db_path)
202     if is_plaintext_sqlite(path):
203         raise EncryptedDatabaseError(
204             f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
encrypt' "
205             "to migrate it to an encrypted ledger"
206         )
207     with connect(path, key=get_or_create_key()) as conn:
208         yield conn
209
210
211     def encrypt_database(db_path: Path | str, key: str) -> int:
212         """Migrate a plaintext SQLite ledger to an encrypted one in place, via
213         ``sqlcipher_export``.
214
215         The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and
216         then
217         securely delete (it still holds the cleartext data). Returns the number of user
218         tables
219         copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext
220         SQLite file.
221         """
222         import sqlcipher3
223
224         path = Path(db_path)
225         if not is_plaintext_sqlite(path):
226             raise EncryptedDatabaseError(
227                 f"{path.name} is not a plaintext SQLite database (already encrypted or
missing)"
228             )
229
230         enc_tmp = path.with_name(path.name + ".enc.tmp")
231         enc_tmp.unlink(missing_ok=True)
232         try:
233             src = sqlcipher3.connect(str(path)) # open the plaintext source (no key)
234             try:
235                 src_tables = src.execute(
236                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
237                 ).fetchone()[0]
238                 target = str(enc_tmp).replace("\\", "/") # SQL string literal: forward
slashes
239                 src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY \"{key}\"")
240                 src.execute("SELECT sqlcipher_export('encrypted')")
241                 src.execute("DETACH DATABASE encrypted")
242             finally:
243                 src.close()
244
245             # Verify the encrypted copy BEFORE the destructive swap: it must open with the
246             key, pass
247             # an integrity check, and carry every table across. (The plaintext .bak is kept
248             either way,
249             # so a failed check is recoverable ? but we refuse to swap in a bad file.)
250             with connect(str(enc_tmp), key=key) as check:
251                 integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
252                 ntables = check.execute(
253                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
254                 ).fetchone()[0]
255             if integrity != "ok" or ntables != src_tables:
256                 raise EncryptedDatabaseError(
257                     f"encryption migration verification failed for {path.name} "
258                     f"(integrity={integrity!r}, tables {ntables}/{src_tables}); original

```

```

left untouched"
253         )
254
255         backup = path.with_name(path.name + ".plaintext.bak")
256         backup.unlink(missing_ok=True)
257         path.rename(backup)
258         enc_tmp.rename(path) # consumed on success
259         _harden_permissions(str(path))
260     finally:
261         # Never leave a partial temp behind: no-op on success (already renamed), cleans
up on error.
262         enc_tmp.unlink(missing_ok=True)
263     return int(ntables)
264
265
266     def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
267         """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).
268
269         Opens with ``current_key`` first (which verifies it), then rekeys. Raises
270         :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
271         muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
272         after this returns successfully.
273         """
274         conn = _open_encrypted(str(db_path), current_key) # validates current_key
275         try:
276             conn.execute(f"PRAGMA rekey = '{new_key}'")
277             conn.commit()
278         finally:
279             conn.close()
280         _harden_permissions(str(db_path))
281
282
283     def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
284         """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).
285
286         Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
287         naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
288         it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
289         """
290         import sqlcipher3
291
292         dest_path = Path(dest)
293         dest_path.parent.mkdir(parents=True, exist_ok=True)
294         dest_path.unlink(missing_ok=True)
295         source = _open_encrypted(str(db_path), key) # validates key
296         try:
297             target = sqlcipher3.connect(str(dest_path))
298             try:
299                 target.execute(_raw_key_pragma(key)) # key the destination so the backup is
encrypted
300                 source.backup(target) # consistent snapshot of the live DB
301             finally:
302                 target.close()
303         finally:
304             source.close()
305         _harden_permissions(str(dest_path))
306
307
308     def init_db(conn: sqlite3.Connection) -> None:
309         """Bring the schema up to date via the migration runner (creates tables on a fresh

```

```

DB).
310
311     The current ``_SCHEMA`` is the frozen *baseline* (version ``SCHEMA_VERSION``);
future changes
312     are forward migrations in :mod:`muneem.migrations`, recorded in
``schema_migrations``.
313     """
314     from muneem import migrations
315
316     migrations.run_migrations(conn, baseline_sql=_SCHEMA,
baseline_version=SCHEMA_VERSION)
317
318
319     def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
320         return conn.execute("SELECT * FROM documents WHERE sha256 = ?",
(sha256,)).fetchone()
321
322
323     def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
324         """Internal helper for the simple INSERT pattern used by documents, offers, and bank
txns."""
325         cols = ", ".join(fields)
326         placeholders = ", ".join("? " for _ in fields)
327         cur = conn.execute(
328             f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
329             tuple(fields.values()),
330         )
331         rowid = cur.lastrowid # always set after a successful INSERT; guard for the type-
checker
332         if rowid is None: # pragma: no cover - defensive; INSERT always sets lastrowid
333             raise RuntimeError(f"INSERT into {table} did not return a row id")
334         return int(rowid)
335
336
337     def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
338         """Insert into documents table. Delegates to the shared helper for DRY."""
339         return _insert_row(conn, "documents", dict(fields))
340
341
342     def record_provenance(
343         conn: sqlite3.Connection,
344         *,
345         document_id: int,
346         source_type: str,
347         parser: str | None = None,
348         parser_version: str | None = None,
349         mapping_layer: str | None = None,
350         confidence: str | None = None,
351     ) -> None:
352         """Upsert the provenance row for a parse (one per document ? see DESIGN §8.7).
353
354         First-class but *not* per transaction row: every txn table FKs ``documents(id)`, so
a single
355         provenance record per document is reachable by join. Idempotent on ``document_id``
(a re-parse
356         overwrites rather than duplicating).
357         """
358         conn.execute(
359             "INSERT INTO provenance "
360             "(document_id, source_type, parser, parser_version, mapping_layer, confidence) "
361             "VALUES (?, ?, ?, ?, ?, ?) "
362             "ON CONFLICT(document_id) DO UPDATE SET "
363             "source_type=excluded.source_type, parser=excluded.parser, "
364             "parser_version=excluded.parser_version, mapping_layer=excluded.mapping_layer, "
365             "confidence=excluded.confidence",

```

```

366         (document_id, source_type, parser, parser_version, mapping_layer, confidence),
367     )
368
369
370     def record_parse(
371         conn: sqlite3.Connection,
372         document_id: int,
373         *,
374         confidence: str,
375         parser: str,
376         source_type: str = "pdf_statement",
377         parser_version: str | None = None,
378         mapping_layer: str = "deterministic",
379     ) -> None:
380         """Record a parse outcome: set ``documents.status`` and upsert provenance (one per
381         parse)."""
382         conn.execute("UPDATE documents SET status = ? WHERE id = ?", (confidence,
383         document_id))
384         record_provenance(
385             conn,
386             document_id=document_id,
387             source_type=source_type,
388             parser=parser,
389             parser_version=parser_version,
390             mapping_layer=mapping_layer,
391             confidence=confidence,
392         )
393
394     def get_provenance(conn: sqlite3.Connection, document_id: int) -> sqlite3.Row | None:
395         return conn.execute("SELECT * FROM provenance WHERE document_id = ?",
396         (document_id,)).fetchone()
397
398
399     def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
400         return _insert_row(conn, "offers", fields)
401
402
403     def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
404         return conn.execute(
405             """
406             SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
407                    d.external_id, d.path
408             FROM offers o
409             JOIN documents d ON d.id = o.document_id
410             ORDER BY o.id
411             """
412         ).fetchall()
413
414
415     def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
416         return _insert_row(conn, "axis_transactions", fields)
417
418
419     def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
420         return _insert_row(conn, "axis_cc_transactions", fields)
421
422
423     def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
424     list[sqlite3.Row]:
425         return conn.execute(
426             "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)

```

```

427     def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
428         return conn.execute(
429             "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
430         ).fetchall()
431
432
433     def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
434         return _insert_row(conn, "hdfc_transactions", fields)
435
436
437     def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
438         return conn.execute(
439             "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
440         ).fetchall()
441
442
443     def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
444         return _insert_row(conn, "au_transactions", fields)
445
446
447     def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
448         return conn.execute(
449             "SELECT * FROM au_transactions ORDER BY id DESC LIMIT ?", (limit,)
450         ).fetchall()
451

```

16. user (2026-06-10T16:14:26.534Z)

```

1     """Disaster recovery for the encrypted ledger ? a passphrase-wrapped recovery file.
2
3     The DB key lives in the OS keychain (fast, full-entropy). If that keychain is ever lost
? OS
4     reinstall, new machine, profile/Credential-Manager corruption ? the ledger **and every
same-key
5     backup** become permanently unrecoverable. This module lets the user wrap that key under
a
6     passphrase they choose, into a recovery file they back up **off-machine**; with the file
*and* the
7     passphrase
8     they can restore the key into a fresh keychain.
9
10    Crypto: **Argon2id** (memory-hard KDF) derives a 256-bit wrapping key from the
passphrase + a random
11    salt; **AES-256-GCM** (authenticated) wraps the DB key. The recovery file holds only
12    ``{version, kdf params, salt, nonce, ciphertext}`` ? never the key, never the
passphrase. A wrong
13    passphrase fails the GCM authentication tag ? a clean error, with no decryption oracle.
14    """
15
16    from __future__ import annotations
17
18    import base64
19    import json
20    import os
21    from pathlib import Path
22    from typing import Any
23
24    from cryptography.exceptions import InvalidTag
25    from cryptography.hazmat.primitives.ciphers.aead import AESGCM
26    from cryptography.hazmat.primitives.kdf.argon2 import Argon2id
27
28    from muneem.errors import MuneemError

```

```

29
30     RECOVERY_VERSION = 1
31     _AAD = b"muneem-recovery-v1" # binds the ciphertext to this format/version
32
33     # Argon2id parameters (memory in KiB). Conservative-but-real; the file stores them so
they can be
34     # raised over time without breaking older recovery files.
35     _TIME_COST = 3
36     _MEMORY_KIB = 64 * 1024 # 64 MiB
37     _LANES = 4
38     _SALT_BYTES = 16
39     _NONCE_BYTES = 12
40     _KEY_BYTES = 32
41     _MIN_PASSPHRASE = 8
42
43
44     class RecoveryError(MuneemError):
45         """A recovery file could not be created, or could not be opened (wrong passphrase /
corrupt)."""
46
47
48     def _derive(passphrase: str, salt: bytes, *, time_cost: int, memory_kib: int, lanes:
int) -> bytes:
49         kdf = Argon2id(
50             salt=salt,
51             length=_KEY_BYTES,
52             iterations=time_cost,
53             lanes=lanes,
54             memory_cost=memory_kib,
55         )
56         return kdf.derive(passphrase.encode("utf-8"))
57
58
59     def wrap_key(db_key: str, passphrase: str) -> dict[str, Any]:
60         """Return a JSON-serialisable recovery payload wrapping ``db_key`` under
``passphrase``."""
61         if not passphrase or len(passphrase) < _MIN_PASSPHRASE:
62             raise RecoveryError(f"recovery passphrase must be at least {_MIN_PASSPHRASE}
characters")
63         salt = os.urandom(_SALT_BYTES)
64         nonce = os.urandom(_NONCE_BYTES)
65         key = _derive(passphrase, salt, time_cost=_TIME_COST, memory_kib=_MEMORY_KIB,
lanes=_LANES)
66         ciphertext = AESGCM(key).encrypt(nonce, db_key.encode("utf-8"), _AAD)
67         return {
68             "version": RECOVERY_VERSION,
69             "kdf": {
70                 "algo": "argon2id",
71                 "time_cost": _TIME_COST,
72                 "memory_kib": _MEMORY_KIB,
73                 "lanes": _LANES,
74             },
75             "salt": base64.b64encode(salt).decode(),
76             "nonce": base64.b64encode(nonce).decode(),
77             "ciphertext": base64.b64encode(ciphertext).decode(),
78         }
79
80
81     def unwrap_key(payload: dict[str, Any], passphrase: str) -> str:
82         """Recover the DB key from a recovery ``payload`` + ``passphrase``. Raises
RecoveryError."""
83         try:
84             salt = base64.b64decode(payload["salt"])
85             nonce = base64.b64decode(payload["nonce"])
86             ciphertext = base64.b64decode(payload["ciphertext"])

```

```

87         kdf = payload.get("kdf", {})
88     except (KeyError, ValueError, TypeError) as exc:
89         raise RecoveryError("recovery file is malformed") from exc
90     key = _derive(
91         passphrase,
92         salt,
93         time_cost=int(kdf.get("time_cost", _TIME_COST)),
94         memory_kib=int(kdf.get("memory_kib", _MEMORY_KIB)),
95         lanes=int(kdf.get("lanes", _LANES)),
96     )
97     try:
98         plain = AESGCM(key).decrypt(nonce, ciphertext, _AAD)
99     except InvalidTag as exc:
100         raise RecoveryError("wrong passphrase or corrupt recovery file") from exc
101     return plain.decode("utf-8")
102
103
104 def write_recovery_file(path: Path | str, db_key: str, passphrase: str) -> Path:
105     """Wrap ``db_key`` under ``passphrase`` and write the recovery file (owner-only
106 perms)."""
107     payload = wrap_key(db_key, passphrase)
108     out = Path(path)
109     out.parent.mkdir(parents=True, exist_ok=True)
110     out.write_text(json.dumps(payload, indent=2), encoding="utf-8")
111     try:
112         os.chmod(out, 0o600)
113     except OSError: # pragma: no cover - best-effort on platforms without POSIX perms
114         pass
115     return out
116
117 def read_recovery_file(path: Path | str, passphrase: str) -> str:
118     """Read a recovery file and return the unwrapped DB key. Raises RecoveryError on any
119 failure."""
120     p = Path(path)
121     try:
122         payload = json.loads(p.read_text(encoding="utf-8"))
123     except (OSError, ValueError) as exc:
124         raise RecoveryError(f"could not read recovery file {p.name}") from exc
125     return unwrap_key(payload, passphrase)

```

17. user (2026-06-10T16:14:26.671Z)

```

1     """Local paths and configuration.
2
3     muneem is local-first: config lives in the OS config dir, data (DB + staging)
4     in the OS data dir. Nothing here touches the repository or the network, and no
5     secrets are stored here ? those go through :mod:`muneem.secrets` (the keychain).
6     """
7
8     from __future__ import annotations
9
10    import tomlib
11    from dataclasses import dataclass, field
12    from pathlib import Path
13    from typing import Any
14
15    from platformdirs import PlatformDirs
16
17    _dirs = PlatformDirs(appname="muneem", appauthor=False)
18
19    CONFIG_DIR = Path(_dirs.user_config_dir)
20    DATA_DIR = Path(_dirs.user_data_dir)
21    CONFIG_FILE = CONFIG_DIR / "config.toml"

```

```

22     DB_PATH = DATA_DIR / "muneem.db"
23     STAGING_DIR = DATA_DIR / "staging"
24
25
26     @dataclass(frozen=True)
27     class Config:
28         """Resolved local paths plus any values from ``config.toml``."""
29
30         config_dir: Path = CONFIG_DIR
31         data_dir: Path = DATA_DIR
32         config_file: Path = CONFIG_FILE
33         db_path: Path = DB_PATH
34         staging_dir: Path = STAGING_DIR
35         values: dict[str, Any] = field(default_factory=dict)
36
37
38     def load_config() -> Config:
39         """Load configuration, reading ``config.toml`` if it exists. Read-only."""
40         values: dict[str, Any] = {}
41         if CONFIG_FILE.is_file():
42             with CONFIG_FILE.open("rb") as fh:
43                 values = tomlib.load(fh)
44         return Config(values=values)
45
46
47     def ensure_dirs() -> None:
48         """Create the local config/data/staging dirs if missing. Never touches the repo."""
49         for directory in (CONFIG_DIR, DATA_DIR, STAGING_DIR):
50             directory.mkdir(parents=True, exist_ok=True)
51

```

18. user (2026-06-10T16:14:32.541Z)

```

1     """Resolve a statement PDF's password for the ingest path ? the hybrid strategy, wired.
2
3     DESIGN.md §8.2 / ROADMAP C.1. Resolution order when a statement is encrypted:
4
5     1. **Candidate dictionary** ? passwords cached in the OS keychain (from earlier
successful
6         unlocks) plus those configured in ``password_rules.yaml`` (see
7         :mod:`muneem.password_rules`).
8     2. **Interactive prompt** ? a hidden ``getpass`` prompt (skipped with
``interactive=False``,
9         e.g. automation / CI).
10
11     A password that works **via the prompt** is cached in the keychain so the next ingest is
12     friction-free. Nothing here logs a password, writes one to the repo, or echoes it; the
cache is
13     the OS keychain (:mod:`muneem.secrets`), never a file.
14     """
15
16     from __future__ import annotations
17
18     import getpass
19     import json
20     from pathlib import Path
21
22     from muneem import secrets
23     from muneem.errors import KeychainError
24     from muneem.password_rules import candidate_passwords
25     from muneem.unlock import is_encrypted, unlock
26
27     # Keychain entry holding a JSON list of known-good statement passwords (the learned
candidates).
28     _CACHE_KEY = "statement_unlock_candidates"

```

```

29     # Bound the cached list so the keychain entry can't grow without limit; most-recently-
used kept.
30     _MAX_CACHED = 20
31
32
33     def _cached_candidates() -> list[str]:
34         raw = secrets.get_secret(_CACHE_KEY)
35         if not raw:
36             return []
37         try:
38             return [p for p in json.loads(raw) if isinstance(p, str) and p]
39         except (ValueError, TypeError):
40             return []
41
42
43     def _remember(password: str) -> None:
44         """Cache a working password in the keychain (MRU order, deduped, capped). Never
logged.
45
46         Best-effort: a keychain write failure is swallowed (the password already unlocked
the file this
47         run); we never crash an ingest over a caching hiccup.
48         """
49         if not password or not password.strip():
50             return
51         current = [p for p in _cached_candidates() if p != password]
52         updated = [password, *current][:_MAX_CACHED] # most-recently-used first, bounded
53         try:
54             secrets.set_secret(_CACHE_KEY, json.dumps(updated))
55         except KeychainError:
56             pass
57
58
59     def resolve_password(pdf_path: Path | str, *, interactive: bool = True) -> str | None:
60         """Return a working password for ``pdf_path``.
61
62         ``""`` means the PDF is not encrypted (no password needed). ``None`` means it is
encrypted and
63         no candidate (or prompt) unlocked it. A prompt-resolved password is cached for next
time.
64         """
65         path = Path(pdf_path)
66         if not is_encrypted(path):
67             return ""
68
69         candidates = [*_cached_candidates(), *candidate_passwords()]
70         prompt_fn = None
71         if interactive:
72
73             def prompt_fn() -> str | None: # noqa: E306 ? local prompt; no echo, never
logged
74                 return getpass.getpass(f"Password for {path.name}: ") or None
75
76         result = unlock(path, candidates=candidates, prompt_fn=prompt_fn)
77         if not result.ok:
78             return None
79         if result.method == "prompt" and result.password:
80             _remember(result.password)
81         return result.password or ""
82

```

19. user (2026-06-10T16:14:32.541Z)

```

1     """Decrypt password-protected statement PDFs (Stage 2).
2

```

```

3     Uses pypdf (BSD-3-Clause) ? permissive, pure-Python. Implements the hybrid
4     password strategy from DESIGN.md § 8.2: try a candidate dictionary first, then
5     fall back to an interactive prompt. The working password is returned so the
6     caller can cache it in the OS keychain (muneem.secrets); **it is never logged**.
7     """
8
9     from __future__ import annotations
10
11     import shutil
12     from collections.abc import Callable, Iterable
13     from dataclasses import dataclass
14     from pathlib import Path
15
16     import pypdf
17
18
19     @dataclass
20     class UnlockResult:
21         ok: bool
22         method: str | None # "plaintext" | "candidate" | "prompt"
23         password: str | None # working password ? do NOT log; None if plaintext/failed
24         output_path: Path | None # decrypted copy, if a destination was given
25
26
27     def is_encrypted(path: Path | str) -> bool:
28         return pypdf.PdfReader(str(path)).is_encrypted
29
30
31     def password_works(path: Path | str, password: str) -> bool:
32         reader = pypdf.PdfReader(str(path))
33         if not reader.is_encrypted:
34             return True
35         try:
36             return reader.decrypt(password) != pypdf.PasswordType.NOT_DECRYPTED
37         except Exception:
38             return False
39
40
41     def write_decrypted(path: Path | str, password: str | None, dest: Path | str) -> Path:
42         """Write a decrypted copy of the PDF to dest (password may be '' for plaintext)."""
43         src = Path(path)
44         dest = Path(dest)
45         dest.parent.mkdir(parents=True, exist_ok=True)
46         reader = pypdf.PdfReader(str(src))
47         if reader.is_encrypted:
48             reader.decrypt(password or "")
49         writer = pypdf.PdfWriter()
50         writer.append(reader)
51         with open(dest, "wb") as fh:
52             writer.write(fh)
53         return dest
54
55
56     def unlock(
57         path: Path | str,
58         candidates: Iterable[str] = (),
59         prompt_fn: Callable[[], str | None] | None = None,
60         max_prompts: int = 3,
61         dest: Path | str | None = None,
62     ) -> UnlockResult:
63         """Hybrid unlock: plaintext? -> candidate dictionary -> interactive prompt.
64
65         If dest is given and unlock succeeds, a decrypted copy is written there. The
66         working password is returned in the result but must never be logged.
67         """

```

```

68     src = Path(path)
69     if not is_encrypted(src):
70         out = Path(shutil.copyfile(src, dest)) if dest else None
71         return UnlockResult(True, "plaintext", None, out)
72
73     tried: set[str] = set()
74     for pw in candidates:
75         if pw in tried:
76             continue
77         tried.add(pw)
78         if password_works(src, pw):
79             out = write_decrypted(src, pw, dest) if dest else None
80             return UnlockResult(True, "candidate", pw, out)
81
82     if prompt_fn is not None:
83         for _ in range(max_prompts):
84             entered = prompt_fn()
85             if not entered:
86                 break
87             if password_works(src, entered):
88                 out = write_decrypted(src, entered, dest) if dest else None
89                 return UnlockResult(True, "prompt", entered, out)
90
91     return UnlockResult(False, None, None, None)
92

```

20. user (2026-06-10T16:14:39.346Z)

```

1     """PDF text extraction (the deterministic, born-digital path).
2
3     Uses **pdfminer.six (MIT)** ? a permissive, commercially-safe, pure-Python text
4     extractor. We deliberately do *not* use PyMuPDF/``fitz``: it is **AGPL-3.0**,
5     incompatible with muneem's commercial-safety principle (see docs/DEPENDENCIES.md).
6
7     NFKC normalization is applied because the real corpus returns ligatures
8     (e.g. U+FB00 '?') that would otherwise break naive matching. Image extraction +
9     OCR (for codes rasterized into banners, e.g. Swiggy's CTSWIGGYUX) is the image
10    tier and lands with the OCR work (pypdfium2 + Tesseract, both permissive).
11    """
12
13    from __future__ import annotations
14
15    import hashlib
16    import unicodedata
17    from dataclasses import dataclass
18    from pathlib import Path
19
20    from pdfminer.high_level import extract_text
21
22
23    @dataclass
24    class Extracted:
25        """The result of reading a PDF's text layer."""
26
27        path: Path
28        sha256: str
29        text: str
30
31
32    def _normalize(text: str) -> str:
33        """NFKC-normalize so ligatures fold to ASCII (U+FB00 '?' -> 'ff')."""
34        return unicodedata.normalize("NFKC", text)
35
36
37    def file_sha256(path: Path | str) -> str:

```

```

38     """Stream the file and return its hex SHA-256 (used for document dedup)."""
39     digest = hashlib.sha256()
40     with open(path, "rb") as fh:
41         for chunk in iter(lambda: fh.read(65536), b''):
42             digest.update(chunk)
43     return digest.hexdigest()
44
45
46 def extract(pdf_path: Path | str, password: str = "") -> Extracted:
47     """Extract and NFKC-normalize the full text layer of a PDF.
48
49     ``password`` is forwarded to pdfminer for encrypted statements and used in-memory
only ? it is
50     never stored or logged. The sha256 is over the file bytes, so it is independent of
the password.
51     """
52     path = Path(pdf_path)
53     text = extract_text(str(path), password=password or "")
54     return Extracted(path=path, sha256=file_sha256(path), text=_normalize(text))
55

```

21. user (2026-06-10T16:14:39.768Z)

```

1     """Local OCR for codes rasterized into banner images (the image tier).
2
3     100% local and permissive ? pypdfium2 (Apache/BSD) renders pages, Tesseract
4     (Apache-2.0) reads them. No paid vision API (see DESIGN.md § 5 rule 9).
5
6     Grounded on the corpus: grayscale render at ~288 DPI (scale 4.0) is what reliably
7     reads stylized banner text. Note OCR is approximate ? e.g. Swiggy's white-on-orange
8     boxed `CTSWIGGYUX` reads as `ICTSWIGGYUX` (a leading box-edge artifact). Callers
9     should treat OCR-derived codes as low-confidence ("verify"), and matching against
10    known codes is fuzzy, never exact (docs/TESTING.md § 6). A purpose-trained model
11    is the eventual upgrade for stylized codes.
12    """
13
14    from __future__ import annotations
15
16    import os
17    import shutil
18    import unicodedata
19    from pathlib import Path
20
21    import pypdfium2 as pdfium
22    import pytesseract
23    from PIL import ImageOps
24
25    # Common install locations, tried only if Tesseract isn't on PATH.
26    _DEFAULT_TESSERACT_PATHS = (
27        r"C:\Program Files\Tesseract-OCR\tesseract.exe", # Windows (winget / UB-Mannheim)
28        "/usr/bin/tesseract", # Linux (apt / dnf)
29        "/usr/local/bin/tesseract", # Linux from source / Intel Homebrew
30        "/opt/homebrew/bin/tesseract", # macOS Apple-silicon Homebrew
31    )
32
33
34    def _resolve_tesseract() -> str | None:
35        """Find the Tesseract binary: env override, then PATH, then common OS locations."""
36        cmd = os.environ.get("MUNEEM_TESSERACT_CMD")
37        if cmd and Path(cmd).exists():
38            return cmd
39        found = shutil.which("tesseract")
40        if found:
41            return found
42        for candidate in _DEFAULT_TESSERACT_PATHS:

```

```

43         if Path(candidate).exists():
44             return candidate
45     return None
46
47
48     def tesseract_available() -> bool:
49         return _resolve_tesseract() is not None
50
51
52     def ocr_pdf(pdf_path: Path | str, scale: float = 4.0) -> str:
53         """Render each page locally and OCR it; returns NFKC-normalized text."""
54         cmd = _resolve_tesseract()
55         if cmd is None:
56             raise RuntimeError(
57                 "Tesseract not found. Install it, or set MUNEEM_TESSERACT_CMD to its path."
58             )
59         pytesseract.pytesseract.tesseract_cmd = cmd
60         pdf = pdfium.PdfDocument(str(pdf_path))
61         parts: list[str] = []
62         try:
63             for page in pdf:
64                 image = ImageOps.grayscale(page.render(scale=scale).to_pil())
65                 parts.append(pytesseract.image_to_string(image))
66         finally:
67             pdf.close()
68         return unicodedata.normalize("NFKC", "\n".join(parts))
69

```

22. user (2026-06-10T16:14:48.760Z)

```

1     """Central, redaction-aware logging.
2
3     Use ``get_logger(__name__)`` instead of the stdlib ``logging.getLogger`` so every muneem
logger
4     carries a :class:`RedactingFilter`. The filter scrubs money-/account-like substrings
from each
5     record as a backstop ? code should still pass already-sanitised context (see
6     :mod:`muneem.redaction`); this is defense-in-depth so an accidental sensitive f-string
cannot leak.
7
8     muneem stays library-style: we attach the filter but do not configure handlers or
levels (the
9     application / CLI owns that), so importing muneem never hijacks the root logger.
10    """
11
12    from __future__ import annotations
13
14    import logging
15
16    from muneem.redaction import redact_text
17
18
19    class RedactingFilter(logging.Filter):
20        """A logging filter that masks sensitive substrings in the formatted message."""
21
22        def filter(self, record: logging.LogRecord) -> bool:
23            try:
24                message = record.getMessage()
25            except Exception: # never drop a record just because formatting failed
26                return True
27            redacted = redact_text(message)
28            if redacted != message:
29                record.msg = redacted
30                record.args = ()
31            return True

```

```

32
33
34     _filtered: set[str] = set()
35
36
37     def get_logger(name: str) -> logging.Logger:
38         """Return a logger that redacts sensitive substrings from its records
39         (idempotent)."""
40         logger = logging.getLogger(name)
41         if name not in _filtered:
42             logger.addFilter(RedactingFilter())
43             _filtered.add(name)
44         return logger

```

23. user (2026-06-10T16:14:49.469Z)

```

1     """Redaction helpers ? keep sensitive data out of logs, exceptions, and stdout.
2
3     muneem aggregates the user's whole financial life, so a balance, account number, or
4     password that
5     slips into a log line or an exception message is a real incident. These helpers produce
6     *safe*
7     representations for diagnostics. They are deliberately conservative: it is better to
8     over-mask a
9     diagnostic string than to leak one.
10    """
11
12    from __future__ import annotations
13
14    import re
15    from pathlib import Path
16
17    # Long digit runs look like account / card / customer numbers.
18    _ACCOUNT_RE = re.compile(r"\d{9,}")
19    # Money-like: Indian or Western grouping with a decimal, or a plain N.NN amount.
20    _AMOUNT_RE = re.compile(r"\d{1,3}(?:,\d{2,3})+(?:\.\d{1,2})?|\d+\.\d{2}")
21    # Digit runs in a filename that may be PII (DOB, account suffix, customer id).
22    _FILENAME_DIGITS_RE = re.compile(r"\d{4,}")
23
24    def redact_filename(path: str | Path) -> str:
25        """Return a file's *name only* (never the full path), with long digit runs masked.
26
27        The full path can expose the user's home/account layout, and a bare basename can
28        still carry PII
29        (Indian statement names are often ``Name_DDMMYYYY.pdf``), so digit runs of 4+ are
30        masked. Keeps
31        enough of the name to be useful in a diagnostic.
32        """
33        name = Path(str(path)).name
34        return _FILENAME_DIGITS_RE.sub(lambda m: "#" * len(m.group()), name)
35
36    def redact_text(text: str) -> str:
37        """Mask account-number-like and money-like substrings in free text (e.g. before
38        logging).
39
40        A backstop for accidental leaks ? code should already pass sanitised context; this
41        catches what
42        slips through. Returns the input unchanged if there is nothing to mask.
43        """
44        if not text:
45            return text
46        masked = _AMOUNT_RE.sub("?amt?", text)

```

```

42         masked = _ACCOUNT_RE.sub("?acct?", masked)
43         return masked
44
45
46     def mask_secret(secret: str | None) -> str:
47         """A non-reversible placeholder for a password/key ? its length only, never the
value."""
48         if not secret:
49             return "?none?"
50         return f"?secret:{len(secret)} chars?"
51

```

24. user (2026-06-10T16:14:53.720Z)

```

1     """The ``muneem`` command-line interface.
2
3     Track A ships the promo spine: ``ingest`` a PDF into the local store and
4     ``offers list`` to read back what was found. ``--ocr`` adds local image OCR for
5     codes rasterized into banners (approximate; flagged for verification). Pipeline
6     subcommands (``fetch``, ``txns``) land with their milestones.
7     """
8
9     from __future__ import annotations
10
11     from contextlib import contextmanager
12     from pathlib import Path
13
14     import typer
15
16     from muneem import __version__
17     from muneem import db as dbmod
18     from muneem.config import load_config
19     from muneem.log import get_logger
20     from muneem.redaction import redact_filename
21
22     _log = get_logger(__name__)
23
24     app = typer.Typer(
25         add_completion=False,
26         help="muneem ? a local-first personal-finance bookkeeper.",
27         no_args_is_help=True,
28     )
29     offers_app = typer.Typer(no_args_is_help=True, help="Work with extracted offers.")
30     app.add_typer(offers_app, name="offers")
31
32     txns_app = typer.Typer(no_args_is_help=True, help="Work with parsed transactions.")
33     app.add_typer(txns_app, name="txns")
34
35     db_app = typer.Typer(no_args_is_help=True, help="Manage the local encrypted ledger.")
36     app.add_typer(db_app, name="db")
37
38
39     @contextmanager
40     def _ledger(db_path: Path):
41         """Open the encrypted ledger, turning an un-migrated *plaintext* file into a clean
CLI
42         error (pointing at ``muneem db encrypt``) rather than a raw traceback."""
43         try:
44             with dbmod.connect_encrypted(db_path) as conn:
45                 yield conn
46         except dbmod.EncryptedDatabaseError as exc:
47             typer.echo(str(exc), err=True)
48             raise typer.Exit(1) from exc
49
50

```

```

51 # Dispatch table for supported banks (extensible as new parsers land).
52 BANK_DISPATCH = {
53     "axis": (dbmod.list_axis_transactions, "debit", "credit"),
54     "axis_cc": (dbmod.list_axis_cc_transactions, "amount", "type"),
55     "au": (dbmod.list_au_transactions, "debit", "credit"),
56     "hdfc": (dbmod.list_hdfc_transactions, "withdrawal", "deposit"),
57 }
58
59
60 def _version_callback(value: bool) -> None:
61     if value:
62         typer.echo(f"muneem {__version__}")
63         raise typer.Exit()
64
65
66 @app.callback()
67 def main(
68     version: bool = typer.Option(
69         False,
70         "--version",
71         callback=_version_callback,
72         is_eager=True,
73         help="Show the muneem version and exit.",
74     ),
75 ) -> None:
76     """muneem command-line interface."""
77
78
79 @app.command()
80 def info() -> None:
81     """Show resolved local paths (no secrets, no sensitive data)."""
82     cfg = load_config()
83     present = "present" if cfg.config_file.is_file() else "absent"
84     typer.echo(f"muneem {__version__}")
85     typer.echo(f"config dir : {cfg.config_dir}")
86     typer.echo(f"data dir   : {cfg.data_dir}")
87     typer.echo(f"db path    : {cfg.db_path}")
88     typer.echo(f"staging   : {cfg.staging_dir}")
89     typer.echo(f"config file: {cfg.config_file} ({present})")
90
91
92 def _ingest_bank_statement(pdf: Path, db_path: Path, doc_type: str, *, interactive:
93 bool) -> None:
94     """Unlock (if encrypted) ? extract ? select parser ? store reconciled transactions.
95
96     The hybrid password unlock (cached/configured candidates ? prompt) runs *before*
97     extraction,
98     since the text needed to pick a parser is itself encrypted. The resolved password is
99     forwarded
100     to extraction and the parser, used in-memory only ? never logged.
101     """
102     from muneem import db as dbmod
103     from muneem.extract import extract
104     from muneem.parsers import get_parser
105     from muneem.statement_unlock import resolve_password
106     from muneem.unlock import is_encrypted
107
108     password: str | None = ""
109     if is_encrypted(pdf):
110         password = resolve_password(pdf, interactive=interactive)
111         if password is None:
112             hint = " (run without --no-prompt to enter it)" if not interactive else ""
113             typer.echo(f"Could not unlock {pdf.name}: no working password{hint}.",
114 err=True)
115         raise typer.Exit(1)

```

```

112
113     ex = extract(pdf, password=password or "") # resolved above; `or ""` keeps the type
str
114     parser = get_parser(ex.text)
115     if parser is None:
116         typer.echo(f"No parser recognizes {pdf.name} as a supported bank statement.",
err=True)
117         raise typer.Exit(1)
118
119     with _ledger(db_path) as conn:
120         dbmod.init_db(conn)
121         existing = dbmod.get_document_by_sha(conn, ex.sha256)
122         if existing is not None:
123             typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
124             return
125         doc_id = dbmod.insert_document(
126             conn,
127             source="file",
128             external_id=pdf.name,
129             doc_type=doc_type,
130             sha256=ex.sha256,
131             path=str(pdf.resolve()),
132             status="parsed",
133         )
134         count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
135         doc = dbmod.get_document_by_sha(conn, ex.sha256)
136         status = doc["status"] if doc else "parsed"
137
138     typer.echo(
139         f"ingested {pdf.name} (doc {doc_id}); parsed {count} transaction(s) "
140         f"via {parser.__class__.__name__} [{status}]"
141     )
142
143
144 def _eval_statement(pdf: Path, *, interactive: bool) -> tuple[str, int, str | None]:
145     """Run one statement through the full parse pipeline into an in-memory DB.
146
147     Returns ``(outcome, row_count, parser_name)`` where ``outcome`` is the document
status
148     (``reconciled`` / ``unreconciled``) or a failure tag (``locked`` / ``no-parser`` /
``error``).
149     Nothing is persisted and no statement value is surfaced ? only the verdict and
counts.
150     """
151     from muneem import db as dbmod
152     from muneem.extract import extract
153     from muneem.parsers import get_parser
154     from muneem.statement_unlock import resolve_password
155     from muneem.unlock import is_encrypted
156
157     password = ""
158     if is_encrypted(pdf):
159         password = resolve_password(pdf, interactive=interactive) or ""
160         if not password:
161             return ("locked", 0, None)
162     try:
163         ex = extract(pdf, password=password)
164         parser = get_parser(ex.text)
165     except Exception as exc: # eval must not crash on one bad file; record the redacted
reason
166         _log.warning(
167             "eval: extract/route failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
168         )
169     return ("error", 0, None)

```

```

170         if parser is None:
171             return ("no-parser", 0, None)
172         name = parser.__class__.__name__
173         try:
174             with dbmod.connect(":memory:") as conn:
175                 dbmod.init_db(conn)
176                 doc_id = dbmod.insert_document(
177                     conn, source="file", external_id=pdf.name, sha256=ex.sha256,
status="parsed"
178                 )
179                 count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
180                 doc = dbmod.get_document_by_sha(conn, ex.sha256)
181                 status = doc["status"] if doc else "parsed"
182         except Exception as exc:
183             _log.warning(
184                 "eval: parse/store failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
185             )
186             return ("error", 0, name)
187         return (status, count, name)
188
189
190     @app.command("eval")
191     def eval_statements(
192         folder: Path = typer.Argument(
193             ...,
194             exists=True,
195             file_okay=False,
196             readable=True,
197             help="A LOCAL folder of statement PDFs to evaluate (e.g. your seed or golden
set).",
198         ),
199         prompt: bool = typer.Option(
200             False,
201             "--prompt/--no-prompt",
202             help="Prompt for a password when cached/configured candidates don't unlock a
file.",
203         ),
204     ) -> None:
205         """Evaluate the parsers against a folder of statements and report the reconcile
rate.
206
207         The seed?golden onboarding loop (see docs/onboarding-a-bank.md): point it at a few
*seed*
208         statements while tuning a profile, then at a *golden* set to confirm. Everything
runs locally in
209         an in-memory DB; only counts / statuses / parser names print ? never statement
values.
210         """
211         pdfs = sorted(folder.glob("*.pdf"))
212         if not pdfs:
213             typer.echo(f"no .pdf files in {folder}")
214             return
215         reconciled = 0
216         for pdf in pdfs:
217             outcome, count, name = _eval_statement(pdf, interactive=prompt)
218             reconciled += outcome == "reconciled"
219             typer.echo(f" {outcome:12} {count:4d} row(s) {name or '?':20} {pdf.name}")
220         typer.echo(f"\n{reconciled}/{len(pdfs)} reconciled")
221
222
223     @app.command()
224     def ingest(
225         pdf: Path = typer.Argument(
226             ..., exists=True, dir_okay=False, readable=True, help="Path to a PDF to ingest."

```

```

227     ),
228     doc_type: str = typer.Option("promo", help="Document category."),
229     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
230     ocr: bool = typer.Option(
231         False, "--ocr", help="Also OCR images locally (Tesseract) for codes in banners."
232     ),
233     no_prompt: bool = typer.Option(
234         False,
235         "--no-prompt",
236         help="Never prompt for a statement password; use only cached/configured
candidates.",
237     ),
238 ) -> None:
239     """Store a PDF's offers (promo) or transactions (bank_statement) in the local DB."""
240     from muneem.extract import extract
241     from muneem.offers import extract_offer
242
243     db_path = db or load_config().db_path
244
245     if doc_type == "bank_statement":
246         _ingest_bank_statement(pdf, db_path, doc_type, interactive=not no_prompt)
247         return
248
249     from muneem import db as dbmod
250
251     ex = extract(pdf)
252     text_fields = extract_offer(ex.text)
253     # (code, source) pairs; text-layer codes are exact, OCR codes are approximate.
254     found: list[tuple[str, str]] = [(c, "text") for c in text_fields.codes]
255     expiry_raw, expiry_type = text_fields.expiry_raw, text_fields.expiry_type
256
257     if ocr:
258         from muneem.ocr import ocr_pdf, tesseract_available
259
260         if not tesseract_available():
261             typer.echo("warning: --ocr requested but Tesseract not found; skipping
OCR.", err=True)
262         else:
263             ocr_fields = extract_offer(ocr_pdf(pdf))
264             known = {c for c, _ in found}
265             for code in ocr_fields.codes:
266                 if code not in known:
267                     found.append((code, "ocr"))
268                 known.add(code)
269             if expiry_raw is None:
270                 expiry_raw, expiry_type = ocr_fields.expiry_raw, ocr_fields.expiry_type
271
272     with _ledger(db_path) as conn:
273         dbmod.init_db(conn)
274         existing = dbmod.get_document_by_sha(conn, ex.sha256)
275         if existing is not None:
276             typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
277             return
278         doc_id = dbmod.insert_document(
279             conn,
280             source="file",
281             external_id=pdf.name,
282             doc_type=doc_type,
283             sha256=ex.sha256,
284             path=str(pdf.resolve()),
285             status="parsed",
286         )
287         for code, src in found:
288             dbmod.insert_offer(

```

```

289         conn,
290         document_id=doc_id,
291         promo_code=code,
292         expiry_raw=expiry_raw,
293         expiry_type=expiry_type,
294         source=src,
295         source_text=ex.text[:2000],
296     )
297
298     summary = ", ".join(f"{c} [{s}]" for c, s in found) if found else "none"
299     typer.echo(f"ingested {pdf.name} (doc {doc_id}); {len(found)} offer(s): {summary}")
300
301
302 @offers_app.command("list")
303 def offers_list(
304     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
305     dir)."),
306     ) -> None:
307     """List extracted offers."""
308     from muneem import db as dbmod
309
310     db_path = db or load_config().db_path
311     if not Path(db_path).exists():
312         typer.echo("no offers yet (database not created).")
313         return
314
315     with _ledger(db_path) as conn:
316         dbmod.init_db(conn)
317         rows = dbmod.list_offers(conn)
318
319     if not rows:
320         typer.echo("no offers found.")
321         return
322
323     for row in rows:
324         who = row["merchant"] or row["external_id"] or "?"
325         when = f" ? expires {row['expiry_raw']}" if row["expiry_raw"] else ""
326         tag = " [ocr ? verify]" if row["source"] == "ocr" else ""
327         typer.echo(f"[{row['id']}] {who}: {row['promo_code']}{when}{tag}")
328
329
330 @txns_app.command("list")
331 def txns_list(
332     bank: str = typer.Argument(..., help="Which bank's transactions to list (e.g. hdfc,
333     axis)."),
334     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
335     dir)."),
336     limit: int = typer.Option(50, help="Max number of transactions to show."),
337     ) -> None:
338     """List parsed transactions for a specific bank."""
339     from muneem import db as dbmod
340
341     db_path = db or load_config().db_path
342     if not Path(db_path).exists():
343         typer.echo("database not created yet.")
344         return
345
346     bank = bank.lower()
347     if bank not in BANK_DISPATCH:
348         typer.echo(f"Unknown bank: {bank}", err=True)
349         return
350
351     list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
352     with _ledger(db_path) as conn:
353         dbmod.init_db(conn)
354         rows = list_fn(conn, limit=limit)

```

```

351
352     if not rows:
353         typer.echo(f"no {bank} transactions found.")
354         return
355     for row in rows:
356         if bank == "axis_cc":
357             amount = f"+{row[pos_key]}" if row["type"] == "Cr" else f"-{row[neg_key]}"
358         else:
359             amount = f"+{row[pos_key]}" if row[pos_key] else f"-{row[neg_key]}"
360         part = (row["particulars"] or "")[:50]
361         typer.echo(f"[{row['account_number']}] {row['txn_date']} : {part:<50} |
{amount:>10}")
362
363
364     @db_app.command("status")
365     def db_status(
366         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
367     ) -> None:
368         """Show the ledger's encryption state (no statement data is read or printed)."""
369         from muneem import db as dbmod
370         from muneem.db_key import peek_key
371
372         db_path = db or load_config().db_path
373         key_state = "present in keychain" if peek_key() else "not yet minted"
374         if not Path(db_path).exists():
375             typer.echo(f"ledger : {db_path} (does not exist yet)")
376             typer.echo(f"key      : {key_state}")
377             return
378         if dbmod.is_plaintext_sqlite(db_path):
379             state = "PLAINTEXT ? run 'muneem db encrypt' to harden it"
380         else:
381             state = "encrypted (SQLCipher)"
382         typer.echo(f"ledger : {db_path}")
383         typer.echo(f"state  : {state}")
384         typer.echo(f"key    : {key_state}")
385
386
387     @db_app.command("encrypt")
388     def db_encrypt(
389         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
390     ) -> None:
391         """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
place."""
392         from muneem import db as dbmod
393         from muneem.db_key import get_or_create_key
394
395         db_path = db or load_config().db_path
396         if not Path(db_path).exists():
397             typer.echo(
398                 f"nothing to encrypt: {db_path} does not exist (a new ledger is born
encrypted)."
399             )
400             return
401         if not dbmod.is_plaintext_sqlite(db_path):
402             typer.echo(
403                 f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
to do."
404             )
405             return
406         n = dbmod.encrypt_database(db_path, get_or_create_key())
407         typer.echo(
408             f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
remains at "

```

```

409         f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
410     )
411
412
413     def _require_encrypted_ledger(db_path: Path) -> str:
414         """Return the ledger's keychain key, or exit with a clear message if it can't be
415         used."""
416         from muneem import db as dbmod
417         from muneem.db_key import peek_key
418
419         if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
420             typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
421             err=True)
422             raise typer.Exit(1)
423         key = peek_key()
424         if not key:
425             typer.echo("no encryption key in the keychain for this ledger.", err=True)
426             raise typer.Exit(1)
427         return key
428
429     @db_app.command("rotate-key")
430     def db_rotate_key(
431         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
432         dir)."),
433     ) -> None:
434         """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
435         from muneem import db as dbmod
436         from muneem import db_key
437
438         db_path = db or load_config().db_path
439         current = _require_encrypted_ledger(db_path)
440         fresh = db_key.new_key()
441         # Keychain-first with rollback: if the file rekey fails, the keychain still matches
442         the file.
443         db_key.set_key(fresh)
444         try:
445             dbmod.rekey_database(db_path, current, fresh)
446         except dbmod.EncryptedDatabaseError as exc:
447             db_key.set_key(current) # roll back; the file is still under the old key
448             typer.echo(str(exc), err=True)
449             raise typer.Exit(1) from exc
450         typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
451
452     @db_app.command("backup")
453     def db_backup(
454         dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
455         copy."),
456         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
457         dir)."),
458     ) -> None:
459         """Write an encrypted online-backup copy of the ledger (consistent even with WAL
460         active)."""
461         from muneem import db as dbmod
462
463         db_path = db or load_config().db_path
464         key = _require_encrypted_ledger(db_path)
465         try:
466             dbmod.backup_database(db_path, dest, key)
467         except dbmod.EncryptedDatabaseError as exc:
468             typer.echo(str(exc), err=True)
469             raise typer.Exit(1) from exc
470         typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
471

```

```

467
468 @db_app.command("setup-recovery")
469 def db_setup_recovery(
470     out: Path | None = typer.Option(
471         None, help="Recovery file path (default: <data dir>/muneem-recovery.json).",
472     ),
473 ) -> None:
474     """Create a passphrase-wrapped recovery file for the ledger's encryption key.
475
476     Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
477     lost. The
478     file holds only the *wrapped* key ? useless without the passphrase.
479     """
480     import getpass
481
482     from muneem import recovery
483     from muneem.db_key import peek_key
484
485     key = peek_key()
486     if not key:
487         typer.echo("no encryption key in the keychain yet (create the ledger first).",
488 err=True)
489         raise typer.Exit(1)
490     out_path = out or (load_config().data_dir / "muneem-recovery.json")
491     passphrase = getpass.getpass("New recovery passphrase: ")
492     if passphrase != getpass.getpass("Confirm passphrase: "):
493         typer.echo("passphrases did not match.", err=True)
494         raise typer.Exit(1)
495     try:
496         recovery.write_recovery_file(out_path, key, passphrase)
497     except recovery.RecoveryError as exc:
498         typer.echo(str(exc), err=True)
499         raise typer.Exit(1) from exc
500     typer.echo(
501         f"wrote recovery file to {out_path}. Store it OFF this machine (password manager
502 / "
503         "separate drive) and remember the passphrase ? you need BOTH to recover. Without
504 them, a "
505         "lost keychain means the ledger and its backups are unrecoverable."
506     )
507
508 @db_app.command("recover")
509 def db_recover(
510     file: Path = typer.Argument(..., exists=True, dir_okay=False, help="Recovery file
511 path."),
512     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
513 dir)."),
514 ) -> None:
515     """Restore the ledger's encryption key from a recovery file + passphrase (into the
516 keychain)."""
517     import getpass
518
519     from muneem import db as dbmod
520     from muneem import db_key, recovery
521
522     passphrase = getpass.getpass("Recovery passphrase: ")
523     try:
524         key = recovery.read_recovery_file(file, passphrase)
525     except recovery.RecoveryError as exc:
526         typer.echo(str(exc), err=True)
527         raise typer.Exit(1) from exc
528     db_path = db or load_config().db_path
529     # If the ledger is present, verify the recovered key actually opens it before
530     committing it.

```

```

524         if Path(db_path).exists() and not dbmod.is_plaintext_sqlite(db_path):
525             try:
526                 with dbmod.connect(db_path, key=key) as conn:
527                     conn.execute("SELECT 1")
528             except dbmod.EncryptedDatabaseError as exc:
529                 typer.echo("recovered a key, but it does not open the current ledger.",
err=True)
530                 raise typer.Exit(1) from exc
531         db_key.set_key(key)
532         typer.echo("recovered the encryption key into the keychain.")
533
534
535     if __name__ == "__main__":
536         app()
537

```

25. user (2026-06-10T16:15:06.797Z)

```

1     """Security guard: no sensitive paths may be tracked in git.
2
3     Encodes DESIGN.md section 5 ("Nothing sensitive ever in git") and docs/TESTING.md
4     section 9 as a failing-build assertion. If this test fails, something that must
5     stay local has been committed.
6     """
7
8     from __future__ import annotations
9
10    import subprocess
11    from pathlib import Path
12
13    REPO_ROOT = Path(__file__).resolve().parents[1]
14
15    FORBIDDEN_PREFIXES = ("testdata/statements/", "testdata/private/")
16    FORBIDDEN_SUFFIXES = (".db", ".sqlite", ".sqlite3", ".env", ".pem", ".key")
17    FORBIDDEN_NAMES = (
18        "token.json",
19        "token.pickle",
20        "credentials.json",
21        "password_rules.yaml",
22        "password_rules.yml",
23        "password_rules.json",
24        "passwords.txt",
25    )
26
27
28    def _tracked_files() -> list[str]:
29        out = subprocess.run(
30            ["git", "ls-files"],
31            cwd=REPO_ROOT,
32            capture_output=True,
33            text=True,
34            check=True,
35        )
36        return [line.strip() for line in out.stdout.splitlines() if line.strip()]
37
38
39    def test_no_sensitive_paths_tracked():
40        offenders = []
41        for path in _tracked_files():
42            low = path.lower()
43            if (
44                low.startswith(FORBIDDEN_PREFIXES)
45                or low.endswith(FORBIDDEN_SUFFIXES)
46                or Path(path).name.lower() in FORBIDDEN_NAMES
47            ):

```

```

48         offenders.append(path)
49         assert not offenders, f"sensitive paths must never be tracked: {offenders}"
50
51
52     def test_only_corpus_pdfs_are_tracked():
53         """Statements are PDFs ? the only PDFs git may track are the non-sensitive promo
corpus.
54
55         This is the guard that would have caught real HDFC statements being committed to
56         testdata/hdfc-statements/.
57         """
58         offenders = [
59             path
60             for path in _tracked_files()
61             if path.lower().endswith(".pdf") and not path.startswith("testdata/promo-
emails/")
62         ]
63         assert not offenders, f"only testdata/promo-emails/ PDFs may be tracked; found:
{offenders}"
64
65
66     def test_no_noncorpus_testdata_is_tracked():
67         """Mirror of the pre-commit guard: under testdata/, only the non-sensitive
68         promo-emails/ and synthetic/ corpora may be tracked ? nothing else (real
69         statements, zips, credentials, snapshots all stay local)."""
70         allowed = ("testdata/promo-emails/", "testdata/synthetic/")
71         offenders = [
72             path
73             for path in _tracked_files()
74             if path.startswith("testdata/") and not path.startswith(allowed)
75         ]
76         assert not offenders, (
77             f"only promo-emails/ and synthetic/ may be tracked under testdata/; found:
{offenders}"
78         )
79

```

26. user (2026-06-10T16:15:06.989Z)

```

1     name: ci
2
3     on:
4         # All-branch pushes so feature branches are gated even before a PR exists
5         # (the regression fixtures in tests/test_regression_synth.py run here, secret-free).
6         push:
7         pull_request:
8         workflow_dispatch:
9
10    jobs:
11        test:
12            runs-on: ${ matrix.os }
13            strategy:
14                fail-fast: false
15            matrix:
16                os: [ubuntu-latest]
17                python-version: ["3.11", "3.12", "3.13"]
18            include:
19                # The dev box is Windows: test it on the primary Python (DPAPI secret backend,
Windows
20                # path handling, the sqlcipher3 win_amd64 wheel). Linux covers the 3.11/3.12
spread.
21                - os: windows-latest
22                  python-version: "3.13"
23        steps:
24            - uses: actions/checkout@v6

```

```

25         - uses: actions/setup-python@v6
26         with:
27             python-version: ${ matrix.python-version }
28             cache: pip
29             cache-dependency-path: pyproject.toml
30         - name: Install
31           run: |
32             python -m pip install --upgrade pip
33             python -m pip install -e '.[dev]'
34         - name: Lint
35           run: ruff check .
36         - name: Format check
37           run: ruff format --check .
38         - name: Type check
39           run: mypy
40         - name: Test
41           run: pytest --cov=muneem --cov-report=term-missing --cov-fail-under=80
42
43     ocr:
44         # OCR tier needs the Tesseract binary; runs on demand (Linux) to avoid per-PR
45         if: github.event_name == 'workflow_dispatch'
46         runs-on: ubuntu-latest
47         steps:
48             - uses: actions/checkout@v6
49             - uses: actions/setup-python@v6
50             with:
51                 python-version: "3.13"
52                 cache: pip
53                 cache-dependency-path: pyproject.toml
54             - name: Install Tesseract
55               run: sudo apt-get update && sudo apt-get install -y tesseract-ocr
56             - name: Install
57               run: |
58                 python -m pip install --upgrade pip
59                 python -m pip install -e '.[dev]'
60             - name: OCR tests
61               run: pytest -m ocr
62

```

27. user (2026-06-10T16:15:14.032Z)

```

1     """The OS-keychain wrappers: graceful read, attributed (value-free) write, swallowed
2     delete.
3
4     The autouse conf test fixture swaps ``muneem.secrets`` for an in-memory store, so we
5     capture the
6
7     *real* wrapper functions at import time and drive them by patching the underlying
8     ``keyring`` lib.
9
10    """
11
12    import keyring
13    import pytest
14    from keyring.errors import KeyringError
15
16    import muneem.secrets as secretsmod
17    from muneem.errors import KeychainError
18
19    # Captured before the autouse keychain fixture rebinds the module attributes per-test.
20    _real_get = secretsmod.get_secret
21    _real_set = secretsmod.set_secret
22    _real_delete = secretsmod.delete_secret
23
24    def test_get_secret_returns_none_when_keyring_unavailable(monkeypatch):

```

```

21     def boom(service, name):
22         raise KeyringError("no backend")
23
24     monkeypatch.setattr(keyring, "get_password", boom)
25     assert _real_get("anything") is None # graceful: absence, not a crash
26
27
28     def test_get_secret_returns_the_value(monkeypatch):
29         monkeypatch.setattr(keyring, "get_password", lambda s, n: "v" if n == "k" else None)
30         assert _real_get("k") == "v"
31         assert _real_get("missing") is None
32
33
34     def test_set_secret_raises_keychain_error_with_name_but_not_value(monkeypatch):
35         def boom(service, name, value):
36             raise KeyringError("locked")
37
38         monkeypatch.setattr(keyring, "set_password", boom)
39         with pytest.raises(KeychainError) as ei:
40             _real_set("db-key", "super-secret-value")
41         msg = str(ei.value)
42         assert "db-key" in msg # the name is safe to surface
43         assert "super-secret-value" not in msg # the value must NEVER appear
44
45
46     def test_delete_secret_swallows_keyring_errors(monkeypatch):
47         def boom(service, name):
48             raise KeyringError("nope")
49
50         monkeypatch.setattr(keyring, "delete_password", boom)
51         _real_delete("k") # must not raise
52

```

28. user (2026-06-10T16:15:14.608Z)

```

1     """Rugged DB invariants ? CI tripwires that block a PR when a *crucial* database
guarantee
2     regresses, **unless the change is made explicit in the same PR**.
3
4     Two kinds of guard:
5
6     1. **Security / integrity invariants** that must ALWAYS hold ? a failure is a real
regression to
7         fix, never something to "allow":
8         - the production ledger is *never* written in plaintext (encryption seam not
bypassed);
9         - a keyed open never silently downgrades to plaintext if the SQLCipher binding is
missing;
10        - foreign keys are enforced (and cascade);
11        - `documents.sha256` dedup is unique.
12
13    2. **Schema lock** ? pins ``SCHEMA_VERSION`` and the exact table/column shape. An
*accidental*
14        schema drift fails CI. A *deliberate* schema change is "explicitly allowed" by
updating
15        ``EXPECTED_SCHEMA`` / ``EXPECTED_SCHEMA_VERSION`` below **in the same PR** (a
visible, reviewable
16        diff) ? ideally paired with a migration. This is the tripwire the request asks for:
schema
17        churn can't sneak through a merge unnoticed.
18
19    All checks are offline, fast, and use fabricated data only.
20    """
21
22    from __future__ import annotations

```

```

23
24     import sqlite3
25     import sys
26
27     import pytest
28
29     from muneem import db as dbmod
30
31     # --- Schema lock
32     -----
33     # Update BOTH of these (and add a migration) when you intentionally change the schema.
34     That edit
35     # is the "explicitly allowed" signal ? it shows up in the PR diff and forces a conscious
36     decision.
37
38     EXPECTED_SCHEMA_VERSION = 5 # baseline v4 + forward migration v5 (provenance)
39
40     # table -> [(column, declared_type, notnull, pk), ...] as reported by PRAGMA table_info.
41     EXPECTED_SCHEMA: dict[str, list[tuple[str, str, int, int]]] = {
42         "au_transactions": [
43             ("id", "INTEGER", 0, 1),
44             ("document_id", "INTEGER", 1, 0),
45             ("account_number", "TEXT", 0, 0),
46             ("txn_date", "TEXT", 0, 0),
47             ("value_date", "TEXT", 0, 0),
48             ("particulars", "TEXT", 0, 0),
49             ("debit", "REAL", 0, 0),
50             ("credit", "REAL", 0, 0),
51             ("balance", "REAL", 0, 0),
52         ],
53         "axis_cc_transactions": [
54             ("id", "INTEGER", 0, 1),
55             ("document_id", "INTEGER", 1, 0),
56             ("account_number", "TEXT", 0, 0),
57             ("txn_date", "TEXT", 0, 0),
58             ("particulars", "TEXT", 0, 0),
59             ("merchant_category", "TEXT", 0, 0),
60             ("amount", "REAL", 0, 0),
61             ("type", "TEXT", 0, 0),
62         ],
63         "axis_transactions": [
64             ("id", "INTEGER", 0, 1),
65             ("document_id", "INTEGER", 1, 0),
66             ("account_number", "TEXT", 0, 0),
67             ("txn_date", "TEXT", 0, 0),
68             ("value_date", "TEXT", 0, 0),
69             ("particulars", "TEXT", 0, 0),
70             ("debit", "REAL", 0, 0),
71             ("credit", "REAL", 0, 0),
72             ("balance", "REAL", 0, 0),
73         ],
74         "documents": [
75             ("id", "INTEGER", 0, 1),
76             ("source", "TEXT", 1, 0),
77             ("external_id", "TEXT", 0, 0),
78             ("sender", "TEXT", 0, 0),
79             ("subject", "TEXT", 0, 0),
80             ("received_date", "TEXT", 0, 0),
81             ("doc_type", "TEXT", 0, 0),
82             ("sha256", "TEXT", 1, 0),
83             ("path", "TEXT", 0, 0),
84             ("status", "TEXT", 1, 0),
85             ("created_at", "TEXT", 1, 0),
86         ],
87         "hdfc_transactions": [

```

```

85         ("id", "INTEGER", 0, 1),
86         ("document_id", "INTEGER", 1, 0),
87         ("account_number", "TEXT", 0, 0),
88         ("txn_date", "TEXT", 0, 0),
89         ("value_date", "TEXT", 0, 0),
90         ("particulars", "TEXT", 0, 0),
91         ("ref", "TEXT", 0, 0),
92         ("withdrawal", "REAL", 0, 0),
93         ("deposit", "REAL", 0, 0),
94         ("closing_balance", "REAL", 0, 0),
95     ],
96     "offers": [
97         ("id", "INTEGER", 0, 1),
98         ("document_id", "INTEGER", 1, 0),
99         ("merchant", "TEXT", 0, 0),
100        ("promo_code", "TEXT", 0, 0),
101        ("description", "TEXT", 0, 0),
102        ("expiry_raw", "TEXT", 0, 0),
103        ("expiry_type", "TEXT", 0, 0),
104        ("source", "TEXT", 1, 0),
105        ("redeemed", "INTEGER", 1, 0),
106        ("source_text", "TEXT", 0, 0),
107    ],
108    # Provenance/confidence per document ? first-class but a SEPARATE table (DESIGN
$8.7), added by
109    # forward migration v5. Txn rows reach it via their documents(id) FK; not duplicated
per row.
110    "provenance": [
111        ("id", "INTEGER", 0, 1),
112        ("document_id", "INTEGER", 1, 0),
113        ("source_type", "TEXT", 1, 0),
114        ("parser", "TEXT", 0, 0),
115        ("parser_version", "TEXT", 0, 0),
116        ("mapping_layer", "TEXT", 0, 0),
117        ("confidence", "TEXT", 0, 0),
118        ("created_at", "TEXT", 1, 0),
119    ],
120    # Bookkeeping for the hand-rolled migration runner (muneem/migrations.py). Present
from the
121    # baseline; not a data table, but pinned here so its shape can't drift silently
either.
122    "schema_migrations": [
123        ("version", "INTEGER", 0, 1),
124        ("name", "TEXT", 1, 0),
125        ("checksum", "TEXT", 1, 0),
126        ("applied_at", "TEXT", 1, 0),
127    ],
128 }
129
130
131 def _schema_fingerprint(conn: sqlite3.Connection) -> dict[str, list[tuple[str, str, int,
int]]]:
132     tables = [
133         r[0]
134         for r in conn.execute(
135             "SELECT name FROM sqlite_master WHERE type='table' AND name NOT LIKE
'sqlite_%' "
136             "ORDER BY name"
137         )
138     ]
139     out: dict[str, list[tuple[str, str, int, int]]] = {}
140     for t in tables:
141         cols = conn.execute(f"PRAGMA table_info({t})").fetchall()
142         # (cid, name, type, notnull, dflt_value, pk) -> (name, TYPE, notnull, pk)
143         out[t] = [(c[1], (c[2] or "").upper(), int(c[3]), int(c[5])) for c in cols]

```

```

144         return out
145
146
147     def test_schema_version_is_locked():
148         with dbmod.connect(":memory:") as conn:
149             dbmod.init_db(conn)
150             actual = conn.execute("PRAGMA user_version").fetchone()[0]
151             assert actual == EXPECTED_SCHEMA_VERSION, (
152                 f"SCHEMA_VERSION changed ({actual} != {EXPECTED_SCHEMA_VERSION}). If
intentional, update "
153                 "EXPECTED_SCHEMA_VERSION here + add a migration (the explicit allow)."
154             )
155
156
157     def test_schema_shape_is_locked():
158         with dbmod.connect(":memory:") as conn:
159             dbmod.init_db(conn)
160             actual = _schema_fingerprint(conn)
161             if actual != EXPECTED_SCHEMA:
162                 diffs: list[str] = []
163                 for t in sorted(set(EXPECTED_SCHEMA) | set(actual)):
164                     if t not in actual:
165                         diffs.append(f" - table REMOVED: {t}")
166                     elif t not in EXPECTED_SCHEMA:
167                         diffs.append(f" - table ADDED: {t} -> {actual[t]}")
168                     elif actual[t] != EXPECTED_SCHEMA[t]:
169                         diffs.append(
170                             f" - table CHANGED: {t}\n          expected {EXPECTED_SCHEMA[t]}\n"
171                             f"          actual   {actual[t]}"
172                         )
173                 raise AssertionError(
174                     "DB schema lock tripped ? the on-disk shape changed:\n"
175                     + "\n".join(diffs)
176                     + "\n\nIf this change is INTENTIONAL: update EXPECTED_SCHEMA (and bump "
177                     + "EXPECTED_SCHEMA_VERSION + add a migration) in this same PR. That edit is
the "
178                     + "explicit allow. If it is NOT intentional, you just caught a regression."
179                 )
180
181
182     # --- Security / integrity invariants (must always hold)
-----
183
184
185     def test_foreign_keys_are_enforced():
186         with dbmod.connect(":memory:") as conn:
187             assert conn.execute("PRAGMA foreign_keys").fetchone()[0] == 1
188
189
190     def test_document_delete_cascades_to_offers():
191         with dbmod.connect(":memory:") as conn:
192             dbmod.init_db(conn)
193             doc = dbmod.insert_document(conn, source="file", sha256="s1", status="parsed")
194             dbmod.insert_offer(conn, document_id=doc, promo_code="X")
195             assert len(dbmod.list_offers(conn)) == 1
196             conn.execute("DELETE FROM documents WHERE id = ?", (doc,))
197             assert dbmod.list_offers(conn) == [] # cascade removed the orphan
198
199
200     def test_documents_sha256_is_unique():
201         with dbmod.connect(":memory:") as conn:
202             dbmod.init_db(conn)
203             dbmod.insert_document(conn, source="file", sha256="dup", status="parsed")
204             with pytest.raises(sqlite3.IntegrityError):
205                 dbmod.insert_document(conn, source="file", sha256="dup", status="parsed")

```

```

206
207
208     def test_cli_ingest_writes_an_encrypted_ledger(tmp_path):
209         """End-to-end guard: the production CLI must write an ENCRYPTED ledger. Catches
anyone
210         swapping the encryption seam (connect_encrypted / _ledger) back to a plaintext
open."""
211         pytest.importorskip("sqlcipher3")
212         from typer.testing import CliRunner
213
214         from muneem.cli import app
215         from synthpdf import at, build_pdf
216
217         pdf = tmp_path / "promo.pdf"
218         build_pdf([at(72, 72, "Use code SAVE20 at checkout")], pdf)
219         led = tmp_path / "ledger.db"
220
221         result = CliRunner().invoke(app, ["ingest", str(pdf), "--db", str(led)])
222         assert result.exit_code == 0, result.output
223         assert led.exists(), "ingest did not create the ledger"
224         assert not dbmod.is_plaintext_sqlite(led), (
225             "production ingest wrote a PLAINTEXT ledger ? the encryption seam was bypassed.
"
226             "Real financial data must never land in cleartext."
227         )
228
229
230     def test_keyed_open_never_silently_downgrades_to_plaintext(tmp_path, monkeypatch):
231         """If the SQLCipher binding is unavailable, a keyed open must RAISE ? never quietly
write a
232         plaintext database in its place."""
233         p = tmp_path / "ledger.db"
234         monkeypatch.setitem(sys.modules, "sqlcipher3", None) # makes `import sqlcipher3`
raise
235         with pytest.raises(dbmod.EncryptedDatabaseError):
236             with dbmod.connect(p, key="00" * 32):
237                 pass
238         assert not dbmod.is_plaintext_sqlite(p), "a plaintext DB must NOT be created as a
fallback"
239

```

29. user (2026-06-10T16:15:19.932Z)

```

1     [build-system]
2     requires = ["setuptools>=61"]
3     build-backend = "setuptools.build_meta"
4
5     [project]
6     name = "muneem"
7     description = "A local-first personal-finance bookkeeper that ingests financial
documents from email into a consolidated ledger."
8     readme = "README.md"
9     requires-python = ">=3.11"
10    dynamic = ["version"]
11    # Private, single-user tool ? this classifier makes packaging tools refuse to upload it
to PyPI.
12    classifiers = ["Private :: Do Not Upload"]
13    dependencies = [
14        "typer>=0.12",
15        "platformdirs>=4.0",
16        "keyring>=25.0",
17        "pdfminer.six>=20240706",
18        "pypdfium2>=4.30",
19        "pytesseract>=0.3.10",
20        "pillow>=10.0",

```

```

21     "pypdf>=4.0",
22     "google-api-python-client>=2.100",
23     "google-auth>=2.30",
24     "google-auth-oauthlib>=1.2",
25     "pdfplumber>=0.11",
26     "sqlcipher3>=0.6",
27     "cryptography>=42",
28 ]
29
30 [project.scripts]
31 muneem = "muneem.cli:app"
32
33 [project.optional-dependencies]
34 outlook = ["msal>=1.0", "requests>=2.0"]
35 dev = [
36     "pytest>=8.0",
37     "pytest-cov>=5.0",
38     "ruff>=0.6",
39     "pyyaml>=6.0",
40     "mypy>=1.8",
41 ]
42
43 [tool.setuptools.dynamic]
44 version = { attr = "muneem.__version__" }
45
46 [tool.setuptools.packages.find]
47 where = ["src"]
48
49 [tool.ruff]
50 line-length = 100
51 src = ["src", "tests"]
52
53 [tool.ruff.lint]
54 select = ["E", "F", "I", "UP", "B"]
55
56 [tool.ruff.format]
57 quote-style = "double"
58 indent-style = "space"
59 skip-magic-trailing-comma = false
60 line-ending = "auto"
61
62 [tool.ruff.lint.flake8-bugbear]
63 # Typer uses Option()/Argument() as parameter defaults by design.
64 extend-immutable-calls = ["typer.Option", "typer.Argument"]
65
66 [tool.mypy]
67 files = ["src/muneem"]
68 python_version = "3.11"
69 ignore_missing_imports = true # not all third-party libs ship type stubs
70 check_untyped_defs = true
71 no_implicit_optional = true
72 warn_redundant_casts = true
73 warn_unused_ignores = true
74 warn_unused_configs = true
75
76 # The email-ingestion providers (gmail/imap/outlook) are experimental scaffolding with
77 # NotImplemented methods (docs/CURRENT_STATE.md §1). Type-harden them when the `fetch`
command
78 # lands (ROADMAP B.4); until then don't gate CI on their half-built signatures.
79 [[tool.mypy.overrides]]
80 module = "muneem.ingestion.*"
81 ignore_errors = true
82
83 [tool.pytest.ini_options]
84 # Default runs are offline and deterministic: the ocr/llm/local_statements tiers

```

```

85     # (see docs/TESTING.md) are opt-in and excluded here.
86     addopts = "-ra -m 'not ocr and not llm and not local_statements'"
87     testpaths = ["tests"]
88     markers = [
89         "ocr: requires the Tesseract binary (image-OCR tier)",
90         "llm: makes live LLM calls (never in default CI)",
91         "local_statements: reads gitignored real statements (local only)",
92     ]
93

```

30. user (2026-06-10T16:15:20.409Z)

```

1     # Convenience targets for Linux/macOS development.
2     # Windows users: use the pip/venv commands in README.md (or run under WSL).
3
4     PYTHON ?= python3
5     VENV    := .venv
6     BIN     := $(VENV)/bin
7
8     .PHONY: install lint format test test-ocr run clean hooks
9
10    # create venv + editable install (with dev deps)
11    $(BIN)/muneem: pyproject.toml
12        $(PYTHON) -m venv $(VENV)
13        $(BIN)/pip install --upgrade pip
14        $(BIN)/pip install -e ".[dev]"
15
16    install: $(BIN)/muneem hooks
17
18    # install the git hooks: pre-commit (blocks committing statements/secrets) and
19    # pre-push (runs lint + the offline test suite, incl. the DB invariant tripwires).
20    hooks:
21        git config core.hooksPath .githooks
22        chmod +x .githooks/pre-commit .githooks/pre-push
23
24    # run ruff (lint + format check)
25    lint: install
26        $(BIN)/ruff check .
27        $(BIN)/ruff format --check .
28
29    # auto-format with ruff (modifies files)
30    format: install
31        $(BIN)/ruff format .
32
33    # run the offline test suite
34    test: install
35        $(BIN)/pytest
36
37    # run OCR tests (needs the tesseract binary)
38    test-ocr: install
39        $(BIN)/pytest -m ocr
40
41    # show resolved local paths
42    run: install
43        $(BIN)/muneem info
44
45    # remove venv and caches
46    clean:
47        rm -rf $(VENV) .pytest_cache .ruff_cache src/muneem.egg-info
48

```

31. user (2026-06-10T16:15:28.930Z)

```

.githooks
exit=0

```

32. user (2026-06-10T16:15:29.368Z)

```
DESIGN.md:21:- Brokerage and mutual-fund accounts (quarterly statements, portfolio snapshots)
DESIGN.md:104:- `holdings` ? point-in-time snapshots of investment positions per account
tests\conf\test.py:30:         snapshot_dir = Path("testdata/snapshots")
tests\test_repo_hygiene.py:69:     statements, zips, credentials, snapshots all stay local)."""
```

33. user (2026-06-10T16:15:34.715Z)

```
1     import json
2     import os
3     from pathlib import Path
4
5     import pytest
6
7
8     @pytest.fixture(autouse=True)
9     def _isolate_keychain(monkeypatch):
10         """Never touch the real OS keychain in tests.
11
12         Replaces :mod:`muneem.secrets` with an in-memory store so the suite cannot pollute
the
13         developer's keychain (e.g. minting a DB key) and runs cleanly in headless CI where
no
14         keyring backend exists. Tests that need specific keychain behaviour re-patch on top.
15         """
16         store: dict[str, str] = {}
17
18         def _set(name, value):
19             store[name] = value
20
21         monkeypatch.setattr("muneem.secrets.get_secret", store.get)
22         monkeypatch.setattr("muneem.secrets.set_secret", _set)
23         monkeypatch.setattr("muneem.secrets.delete_secret", lambda name: store.pop(name,
None))
24         return store
25
26
27     @pytest.fixture
28     def snapshot():
29         def _assert_snapshot(name: str, data: list | dict):
30             snapshot_dir = Path("testdata/snapshots")
31             snapshot_dir.mkdir(parents=True, exist_ok=True)
32             snapshot_file = snapshot_dir / f"{name}.json"
33
34             if os.environ.get("UPDATE_SNAPSHOTS") == "1":
35                 with snapshot_file.open("w", encoding="utf-8") as f:
36                     json.dump(data, f, indent=2, ensure_ascii=False)
37                 return True
38
39             if not snapshot_file.exists():
40                 pytest.fail(f"Snapshot not found at {snapshot_file}. Run with
UPDATE_SNAPSHOTS=1.")
41
42             with snapshot_file.open("r", encoding="utf-8") as f:
43                 expected = json.load(f)
44
45             assert data == expected, f"Data does not match snapshot {name}"
46             return True
47
48         return _assert_snapshot
49
```

34. user (2026-06-10T16:15:42.826Z)

```

1      """Local-only checks for the AU Bank parser against real encrypted statements.
2
3      Skipped unless an AU password is available (`password_rules.yaml` or
4      ``MUNEEM_TEST_AU_PW``); never
5      runs in CI. The parser is **oracle-gated**, so this asserts the safety invariant: it
6      either stored
7      nothing (a vintage it can't yet read) or everything it stored reconciled ? never
8      mislabeled rows.
9      Surfaces counts / booleans / parser names only ? no statement content.
10     """
11
12     import os
13     from pathlib import Path
14
15     import pdfplumber
16     import pytest
17
18     from muneem import db as dbmod
19     from muneem.parsers import get_parser
20     from muneem.password_rules import get_password_for_bank
21
22     AU_PDFS = sorted(Path("testdata/au-bank-statements").glob("*.pdf"))
23
24     @pytest.mark.local_statements
25     @pytest.mark.parametrize("pdf_path", AU_PDFS, ids=lambda p: p.name)
26     def test_local_au_never_stores_unreconciled(pdf_path, tmp_path):
27         pw = (
28             get_password_for_bank("aubank")
29             or get_password_for_bank("au")
30             or os.environ.get("MUNEEM_TEST_AU_PW")
31         )
32         if not pw:
33             pytest.skip("needs the AU password (password_rules.yaml or MUNEEM_TEST_AU_PW)")
34
35         with pdfplumber.open(pdf_path, password=pw) as pdf:
36             text = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(2,
37 len(pdf.pages))))
38             parser = get_parser(text)
39             if parser is None or type(parser).__name__ != "AUParser":
40                 pytest.skip(f"{pdf_path.name}: not routed to AUParser")
41
42             with dbmod.connect(tmp_path / "t.db") as conn:
43                 dbmod.init_db(conn)
44                 doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
45 status="parsed")
46                 n = parser.parse_and_insert(pdf_path, conn, doc_id, password=pw)
47                 status = dbmod.get_document_by_sha(conn, pdf_path.name)["status"]
48                 stored = dbmod.list_au_transactions(conn)
49
50             assert (n == 0 and stored == []) or status == "reconciled", (
51                 f"{pdf_path.name}: stored {n} row(s) with status={status} ? must not persist
52 unreconciled"
53             )

```

35. user (2026-06-10T16:15:43.008Z)

```

1      """Redaction helpers keep sensitive data out of logs / exceptions / stdout."""
2
3      from pathlib import Path
4
5      from muneem.redaction import mask_secret, redact_filename, redact_text
6
7

```

```

8     def test_redact_filename_drops_path_and_masks_pii_digits():
9         out = redact_filename(Path("/home/avi/statements/Avi_Dullu_28022026.pdf"))
10        assert "/" not in out and "\\" not in out # no directory components
11        assert "statements" not in out and "home" not in out
12        assert "28022026" not in out # DOB-like digit run masked
13        assert out.startswith("Avi_Dullu_") and out.endswith(".pdf") # still recognisable
14        assert "#" in out
15
16
17    def test_redact_text_masks_account_and_amount():
18        assert "?acct?" in redact_text("balance for 123456789012 today")
19        assert "?amt?" in redact_text("debit 1,23,456.78 INR") # Indian grouping
20        assert "?amt?" in redact_text("amount 1234.56")
21        assert "123456789012" not in redact_text("acct 123456789012")
22
23
24    def test_redact_text_leaves_clean_text_untouched():
25        assert redact_text("opened 6 files for 2 banks") == "opened 6 files for 2 banks"
26        assert redact_text("") == ""
27
28
29    def test_mask_secret_reveals_length_only():
30        assert mask_secret("hunter2") == "?secret:7 chars?"
31        assert mask_secret("") == "?none?"
32        assert mask_secret(None) == "?none?"
33

```

36. user (2026-06-10T16:15:51.293Z)

```

1    """Local-only checks for the Axis savings parser against real encrypted statements.
2
3    Skipped unless an Axis password is available (`password_rules.yaml` or
4    `MUNEEM_TEST_AXIS_PW`); never runs in CI. The savings parser is oracle-gated
5    (coordinate clustering + `extract_tables` ? engine mapping ? reconciliation), so this
6    asserts the safety invariant: it either stored nothing (a vintage it can't yet read) or
7    everything it stored reconciled ? never mislabeled rows. Surfaces counts / booleans /
8    parser names only ? no statement content.
9    """
10
11    import os
12    from pathlib import Path
13
14    import pdfplumber
15    import pytest
16
17    from muneem import db as dbmod
18    from muneem.parsers import get_parser
19    from muneem.password_rules import get_password_for_bank
20
21    AXIS_PDFS = sorted(Path("testdata/axis-statements").glob("*.pdf"))
22
23
24    @pytest.mark.local_statements
25    @pytest.mark.parametrize("pdf_path", AXIS_PDFS, ids=lambda p: p.name)
26    def test_local_axis_never_stores_unreconciled(pdf_path, tmp_path):
27        pw = get_password_for_bank("axis") or os.environ.get("MUNEEM_TEST_AXIS_PW")
28        if not pw:
29            pytest.skip("needs the Axis password (password_rules.yaml or
30            MUNEEM_TEST_AXIS_PW)")
31
32        with pdfplumber.open(pdf_path, password=pw) as pdf:
33            text = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(2,
34            len(pdf.pages))))
35            parser = get_parser(text)
36            if parser is None:

```

```

35         pytest.skip(f"{pdf_path.name}: no parser recognizes it yet (Axis rewrite
pending)")
36
37     with dbmod.connect(tmp_path / "t.db") as conn:
38         dbmod.init_db(conn)
39         doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
status="parsed")
40         n = parser.parse_and_insert(pdf_path, conn, doc_id, password=pw)
41         status = dbmod.get_document_by_sha(conn, pdf_path.name)["status"]
42         stored = dbmod.list_axis_transactions(conn)
43
44         # Oracle-gated invariant: the savings parser either stores nothing (and flags the
45         # document unreconciled) or, if it stored rows, they reconciled. Never garbage.
46         if type(parser).__name__ == "AxisParser":
47             assert (n == 0 and stored == []) or status == "reconciled", (
48                 f"{pdf_path.name}: stored {n} row(s) with status={status} ? "
49                 "stop-gap must not persist unreconciled rows"
50             )
51

```

37. user (2026-06-10T16:15:51.919Z)

```

1     """Authorize Gmail (read-only) and list the statement emails found.
2
3     Run by *you* ? it opens a browser for Google consent, caches the token in your OS
4     keychain, then prints the statement emails it can see. Nothing is uploaded; this
5     only reads your inbox. Use it to confirm your OAuth client + consent + search work
6     before the full import pipeline.
7
8     python tools/gmail_auth.py --credentials testdata/credentials/client_secret_*.json
9     """
10
11     from __future__ import annotations
12
13     import argparse
14     import glob
15     import sys
16     from pathlib import Path
17
18
19     def _find_credentials(arg: str | None) -> Path | None:
20         if arg:
21             return Path(arg)
22         matches = sorted(glob.glob("testdata/credentials/*.json"))
23         return Path(matches[0]) if matches else None
24
25
26     def main() -> int:
27         parser = argparse.ArgumentParser(
28             description="Authorize Gmail (read-only) and list statement emails."
29         )
30         parser.add_argument("--credentials", help="Path to the OAuth client secret JSON.")
31         parser.add_argument("--sender", default="hdfcbank.net", help="Sender email domain to
search.")
32         parser.add_argument("--max", type=int, default=25, help="Max emails to list.")
33         args = parser.parse_args()
34
35         creds_path = _find_credentials(args.credentials)
36         if not creds_path or not creds_path.exists():
37             print(
38                 "No OAuth client secret found. Pass --credentials <path>. "
39                 "(Create a Desktop OAuth client in Google Cloud Console and download the
JSON; "
40                 "see README § Getting started.)",
41                 file=sys.stderr,

```

```

42         )
43         return 2
44
45     from muneem.ingestion.base import SearchCriteria
46     from muneem.ingestion.gmail import SCOPES, GmailProvider
47
48     print(f"Client secret : {creds_path.name}")
49     print(f"Scope          : {SCOPES[0]} (read-only ? cannot modify or delete mail)")
50
51     provider = GmailProvider(creds_path)
52     provider.authorize()
53
54     criteria = SearchCriteria(sender=args.sender, has_attachment=True,
newer_than_days=None)
55     print(f"Searching      : from:{args.sender} has:attachment")
56
57     ids = provider.search_messages(criteria, max_results=args.max)
58     print(f"Found {len(ids)} message(s) with a PDF attachment:\n")
59     for mid in ids:
60         hdrs = provider.get_headers(mid)
61         attachments = ", ".join(name for name, _ in provider.list_pdf_attachments(mid))
or "(none)"
62         date = hdrs.get("date", "")[:16]
63         subject = hdrs.get("subject", "(no subject)")[:55]
64         print(f" {date:16} {subject:55} {attachments}")
65     print("\nToken cached in your OS keychain. Done.")
66     return 0
67
68
69 if __name__ == "__main__":
70     raise SystemExit(main())
71

```

38. user (2026-06-10T16:15:56.700Z)

```

1     from __future__ import annotations
2
3     import base64
4     import json
5     from pathlib import Path
6
7     from .base import EmailProvider, SearchCriteria
8
9     SCOPES = ["https://www.googleapis.com/auth/gmail.readonly"]
10    _TOKEN_KEY = "gmail_token"
11
12
13    class GmailProvider(EmailProvider):
14        def __init__(self, credentials_path: Path | str, open_browser: bool = True):
15            self.credentials_path = credentials_path
16            self.open_browser = open_browser
17            self.service = None
18
19        def authorize(self) -> None:
20            from google.auth.transport.requests import Request
21            from google.oauth2.credentials import Credentials
22            from google_auth_oauthlib.flow import InstalledAppFlow
23            from googleapiclient.discovery import build
24
25            from muneem import secrets
26
27            creds = None
28            raw = secrets.get_secret(_TOKEN_KEY)
29            if raw:
30                creds = Credentials.from_authorized_user_info(json.loads(raw), SCOPES)

```

```

31         if creds and creds.valid:
32             pass
33         elif creds and creds.expired and creds.refresh_token:
34             creds.refresh(Request())
35         else:
36             flow = InstalledAppFlow.from_client_secrets_file(str(self.credentials_path),
SCOPES)
37             creds = flow.run_local_server(port=0, open_browser=self.open_browser)
38             secrets.set_secret(_TOKEN_KEY, creds.to_json())
39             self.service = build("gmail", "v1", credentials=creds, cache_discovery=False)
40
41     def _build_query(self, criteria: SearchCriteria) -> str:
42         parts = []
43         if criteria.sender:
44             parts.append(f"from:{criteria.sender}")
45         if criteria.has_attachment:
46             parts.append("has:attachment")
47         if criteria.newer_than_days:
48             parts.append(f"newer_than:{criteria.newer_than_days}d")
49         return " ".join(parts)
50
51     def search_messages(self, criteria: SearchCriteria, max_results: int = 100) ->
list[str]:
52         if not self.service:
53             self.authorize()
54
55         query = self._build_query(criteria)
56         ids: list[str] = []
57         page_token = None
58         while True:
59             resp = (
60                 self.service.users()
61                 .messages()
62                 .list(userId="me", q=query, pageToken=page_token,
maxResults=min(max_results, 100))
63                 .execute()
64             )
65             ids.extend(m["id"] for m in resp.get("messages", []))
66             page_token = resp.get("nextPageToken")
67             if not page_token or len(ids) >= max_results:
68                 break
69         return ids[:max_results]
70
71     def _get_message(self, message_id: str) -> dict:
72         if not self.service:
73             self.authorize()
74         return (
75             self.service.users().messages().get(userId="me", id=message_id,
format="full").execute()
76         )
77
78     def get_headers(self, message_id: str) -> dict[str, str]:
79         message = self._get_message(message_id)
80         return {
81             h.get("name", "").lower(): h.get("value", "")
82             for h in message.get("payload", {}).get("headers", [])
83         }
84
85     def _walk_parts(self, payload: dict):
86         stack = [payload]
87         while stack:
88             part = stack.pop()
89             yield part
90             stack.extend(part.get("parts", []))
91

```

```

92     def list_pdf_attachments(self, message_id: str) -> list[tuple[str, str]]:
93         message = self._get_message(message_id)
94         out: list[tuple[str, str]] = []
95         for part in self._walk_parts(message.get("payload", {})):
96             filename = part.get("filename") or ""
97             att_id = part.get("body", {}).get("attachmentId")
98             if att_id and filename.lower().endswith(".pdf"):
99                 out.append((filename, att_id))
100         return out
101
102     def download_attachment(self, message_id: str, attachment_id: str, dest: Path) ->
Path:
103         if not self.service:
104             self.authorize()
105         att = (
106             self.service.users()
107             .messages()
108             .attachments()
109             .get(userId="me", messageId=message_id, id=attachment_id)
110             .execute()
111         )
112         dest = Path(dest)
113         dest.parent.mkdir(parents=True, exist_ok=True)
114         dest.write_bytes(base64.urlsafe_b64decode(att["data"].encode()))
115         return dest
116

```

39. user (2026-06-10T16:16:14.284Z)

```

1     """muneem's exception hierarchy.
2
3     Every error muneem raises derives from :class:`MuneemError`, so callers can catch the
whole family
4     with one `except`. Two rules for messages on these exceptions, because they may end up
in logs:
5
6     1. **Only non-sensitive context.** A file *name* is acceptable (it's the user's own
machine), but
7         never raw statement text, passwords, balances, or account numbers. Use
8         :mod:`muneem.redaction` helpers to sanitise anything derived from a document.
9     2. **Chain the cause.** Raise with ``raise ParseError(...) from exc`` so the original
traceback is
10        available for local debugging while the muneem-level message stays clean.
11    """
12
13    from __future__ import annotations
14
15
16    class MuneemError(Exception):
17        """Base class for all errors raised by muneem."""
18
19
20    class ParseError(MuneemError):
21        """A document could not be parsed (corrupt PDF, unexpected layout, extraction
failure).
22
23        Carries only non-sensitive context (e.g. the file name and the failing stage) ?
never the
24        statement contents or the underlying library's message verbatim.
25        """
26
27
28    class UnlockError(MuneemError):
29        """A password-protected document could not be unlocked (no working password, bad
input)."""

```

```

30
31
32     class EncryptedDatabaseError(MuneemError):
33         """The encrypted ledger could not be opened ? wrong key, missing SQLCipher binding,
or a
34         plaintext file where an encrypted one was expected."""
35
36
37     class KeychainError(MuneemError):
38         """Storing a secret in the OS keychain failed (no backend, locked keychain, etc.).
39
40         The secret *name* is safe to include; never the value."""
41
42
43     class ConfigError(MuneemError):
44         """A user config file (e.g. password_rules.yaml) is malformed.
45
46         Messages must reference the file by name only ? never echo its contents, which may
hold
47         password material."""
48
49
50     class MigrationError(MuneemError):
51         """A schema migration could not be applied, or the on-disk schema diverged from the
52         migration history (e.g. the frozen baseline was edited without adding a
migration)."""
53

```

40. user (2026-06-10T16:16:15.375Z)

Normalize text files to LF in the repo (working tree may differ per-OS).

* text=auto eol=lf

Treat known binary assets as binary so git never tries to munge line endings.

```

*.pdf binary
*.png binary
*.jpg binary
*.jpeg binary
*.gif binary
*.db binary
*.sqlite binary
*.sqlite3 binary

```

41. user (2026-06-10T16:16:16.719Z)

```

src\muneem\cli.py:47:         typer.echo(str(exc), err=True)
src\muneem\cli.py:62:         typer.echo(f"muneem {__version__}")
src\muneem\cli.py:84:         typer.echo(f"muneem {__version__}")
src\muneem\cli.py:85:         typer.echo(f"config dir : {cfg.config_dir}")
src\muneem\cli.py:86:         typer.echo(f"data dir   : {cfg.data_dir}")
src\muneem\cli.py:87:         typer.echo(f"db path   : {cfg.db_path}")
src\muneem\cli.py:88:         typer.echo(f"staging   : {cfg.staging_dir}")
src\muneem\cli.py:89:         typer.echo(f"config file: {cfg.config_file} ({present})")
src\muneem\cli.py:110:         typer.echo(f"Could not unlock {pdf.name}: no working
password{hint}.", err=True)
src\muneem\cli.py:116:         typer.echo(f"No parser recognizes {pdf.name} as a supported bank
statement.", err=True)
src\muneem\cli.py:123:         typer.echo(f"already ingested (doc {existing['id']}):
{pdf.name}")
src\muneem\cli.py:138:         typer.echo(
src\muneem\cli.py:166:             _log.warning(
src\muneem\cli.py:183:             _log.warning(
src\muneem\cli.py:213:         typer.echo(f"no .pdf files in {folder}")

```

```

src\muneem\cli.py:219:         typer.echo(f" {outcome:12} {count:4d} row(s) {name or '?':20}
{pdf.name}")
src\muneem\cli.py:220:     typer.echo(f"\n{reconciled}/{len(pdfs)} reconciled")
src\muneem\cli.py:261:         typer.echo("warning: --ocr requested but Tesseract not found;
skipping OCR.", err=True)
src\muneem\cli.py:276:         typer.echo(f"already ingested (doc {existing['id']}):
{pdf.name}")
src\muneem\cli.py:299:     typer.echo(f"ingested {pdf.name} (doc {doc_id}); {len(found)}
offer(s): {summary}")
src\muneem\cli.py:311:         typer.echo("no offers yet (database not created).")
src\muneem\cli.py:319:         typer.echo("no offers found.")
src\muneem\cli.py:325:         typer.echo(f"[{row['id']}] {who}: {row['promo_code']}{when}{tag}")
src\muneem\cli.py:339:         typer.echo("database not created yet.")
src\muneem\cli.py:344:         typer.echo(f"Unknown bank: {bank}", err=True)
src\muneem\cli.py:353:         typer.echo(f"no {bank} transactions found.")
src\muneem\cli.py:361:         typer.echo(f"[{row['account_number']}] {row['txn_date']} :
{part:<50} | {amount:>10}")
src\muneem\cli.py:375:         typer.echo(f"ledger : {db_path} (does not exist yet)")
src\muneem\cli.py:376:         typer.echo(f"key      : {key_state}")
src\muneem\cli.py:382:     typer.echo(f"ledger : {db_path}")
src\muneem\cli.py:383:     typer.echo(f"state  : {state}")
src\muneem\cli.py:384:     typer.echo(f"key    : {key_state}")
src\muneem\cli.py:397:         typer.echo(
src\muneem\cli.py:402:         typer.echo(
src\muneem\cli.py:407:     typer.echo(
src\muneem\cli.py:419:         typer.echo("no encrypted ledger here (run 'muneem db encrypt'
first).", err=True)
src\muneem\cli.py:423:         typer.echo("no encryption key in the keychain for this ledger.",
err=True)
src\muneem\cli.py:445:         typer.echo(str(exc), err=True)
src\muneem\cli.py:447:     typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
src\muneem\cli.py:463:         typer.echo(str(exc), err=True)
src\muneem\cli.py:465:     typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
src\muneem\cli.py:486:         typer.echo("no encryption key in the keychain yet (create the
ledger first).", err=True)
src\muneem\cli.py:491:         typer.echo("passphrases did not match.", err=True)
src\muneem\cli.py:496:         typer.echo(str(exc), err=True)
src\muneem\cli.py:498:     typer.echo(
src\muneem\cli.py:520:         typer.echo(str(exc), err=True)
src\muneem\cli.py:529:         typer.echo("recovered a key, but it does not open the current
ledger.", err=True)
src\muneem\cli.py:532:     typer.echo("recovered the encryption key into the keychain.")
src\muneem\log.py:1: """Central, redaction-aware logging.
src\muneem\log.py:3: Use ``get_logger(__name__)`` instead of the stdlib ``logging.getLogger`` so
every muneem logger
src\muneem\log.py:9: application / CLI owns that), so importing muneem never hijacks the root
logger.
src\muneem\log.py:19: class RedactingFilter(logging.Filter):
src\muneem\log.py:22:     def filter(self, record: logging.LogRecord) -> bool:
src\muneem\log.py:37: def get_logger(name: str) -> logging.Logger:
src\muneem\log.py:39:     logger = logging.getLogger(name)
src\muneem\log.py:41:     logger.addFilter(RedactingFilter())
src\muneem\ingestion\outlook.py:51:         print(flow["message"]) # Instructs user to go to a
URL and enter code

```

42. user (2026-06-10T16:16:25.044Z)

```

1     """The verifier ? reconciliation as a layout-independent truth oracle.
2
3     Reconciliation proves a parse is complete and correct without inspecting any sensitive
value1:
4     if a transaction is missed or misread, the arithmetic stops adding up. That is what lets
us trust a
5     parse of a statement we never eyeball, and (later) what makes a probabilistic extractor
safe on

```

```

6      money ? *extraction proposes, reconciliation disposes* (see
7      [`docs/parser-architecture.md`](../.../docs/parser-architecture.md) §2, §5).
8
9      This module grew from a single running-balance walk into the verifier the architecture
calls for:
10
11      * **per-row walk** ? ``reconcile_running_balance``: ``balance[i] == balance[i-1] ? debit
+ credit``.
12      * **total reconcile** ? ``reconcile_total``: ``opening + ?credit ? ?debit == closing``
(work even
13      with no per-row balance, e.g. Axis savings).
14      * **bookend** ? ``verify``: parsed Dr/Cr counts + opening/closing vs the bank's printed
SUMMARY.
15      * **selection** ? ``select_reconciling``: among candidate concept mappings, pick the one
arithmetic
16      endorses (the seam the L2 mapping search plugs into).
17      * **confidence** ? ``Confidence`` + ``BookendReport.confidence``: a recorded verdict,
not an
18      assumption.
19
20      Reports carry counts / booleans / a verdict only ? **never an amount**.
21      ""
22
23      from __future__ import annotations
24
25      import re
26      from collections.abc import Sequence
27      from dataclasses import dataclass
28      from enum import StrEnum
29
30      from .concepts import ConceptRow, StatementConcepts
31
32      _CR_DR_SUFFIX = re.compile(r"(?i)\s*(cr|dr)\s*$")
33
34
35      def parse_amount(value: object) -> float | None:
36          """Parse a statement money cell to a float.
37
38          Returns ``None`` only for a genuinely absent cell (None / empty / ``"-."`` /
39          non-numeric); ``"0.00"`` parses to ``0.0``. This deliberately distinguishes
40          *absent* from *zero* ? unlike ``base.safe_amount``, which maps ``"0.00"`` to
41          ``None`` and would corrupt a real zero balance and the reconciliation math.
42          ""
43          if value is None:
44              return None
45          text = _CR_DR_SUFFIX.sub("", str(value).strip()).replace(",", "").strip()
46          if text in ("", "-"):
47              return None
48          try:
49              return float(text)
50          except ValueError:
51              return None
52
53
54      @dataclass
55      class ReconcileReport:
56          """Result of a running-balance walk. Counts only, never an amount."""
57
58          n: int # rows considered
59          compared: int # consecutive (balance-bearing) pairs actually checked
60          breaks: int
61          first_break: int | None = None
62
63          @property
64          def ok(self) -> bool:

```

```

65         return self.compared > 0 and self.breaks == 0
66
67
68     def reconcile_running_balance(
69         entries: Sequence[tuple[float | None, float | None, float | None]],
70         tol: float = 0.01,
71     ) -> ReconcileReport:
72         """Verify ``balance[i] == balance[i-1] - debit[i] + credit[i]`` across rows.
73
74         ``entries`` is an ordered list of ``(debit, credit, balance)``; ``debit`` and
75         ``credit`` may be ``None`` (treated as 0.0). A row whose ``balance`` is ``None``
76         is skipped (can't anchor on it). Because the walk spans the whole statement, a
77         clean result also rules out any row missed *between* two balances ? which is the
78         failure mode silent ``extract_tables()`` row-drops would otherwise hide.
79         """
80         breaks = 0
81         first_break: int | None = None
82         compared = 0
83         prev: float | None = None
84         for i, (debit, credit, balance) in enumerate(entries):
85             if balance is None:
86                 continue
87             if prev is not None:
88                 expected = prev - (debit or 0.0) + (credit or 0.0)
89                 compared += 1
90                 if abs(expected - balance) > tol:
91                     breaks += 1
92                     if first_break is None:
93                         first_break = i
94             prev = balance
95         return ReconcileReport(
96             n=len(entries), compared=compared, breaks=breaks, first_break=first_break
97         )
98
99
100     def _row_entries(
101         rows: Sequence[ConceptRow],
102     ) -> list[tuple[float | None, float | None, float | None]]:
103         """Project concept rows onto the ``(debit, credit, balance)`` tuples the walk
104         consumes."""
105         return [(r.debit, r.credit, r.balance) for r in rows]
106
107
108     def reconcile_total(
109         rows: Sequence[ConceptRow], opening: float, closing: float, tol: float = 0.01
110     ) -> bool:
111         """Verify the layout-independent identity ``opening + ?credit ? ?debit == closing``.
112
113         This is the oracle for statements with **no reliable per-row balance** (e.g. Axis
114         savings):
115         it checks the whole statement against its own opening/closing bookends without
116         needing each
117         row's running balance. ``opening`` and ``closing`` come from the bank's
118         SUMMARY/header
119         (``StatementConcepts``); the caller must have them.
120         """
121         delta = sum(r.credit or 0.0 for r in rows) - sum(r.debit or 0.0 for r in rows)
122         return abs((opening + delta) - closing) <= tol
123
124
125     class Confidence(StrEnum):
126         """Recorded trust verdict for a parsed statement (mirrors ``documents.status``
127         values).
128
129         ``RECONCILED`` ? the parse passed every available check, so it is trusted.

```

```

125     ``UNRECONCILED`` ?
126     a check failed (or none could run); the parse is suspect and is quarantined rather
127     than trusted.
128     """
129     RECONCILED = "reconciled"
130     UNRECONCILED = "unreconciled"
131
132     @dataclass
133     class BookendReport:
134         """A parse cross-checked against the bank's own SUMMARY. Booleans + counts only.
135
136         ``None`` for a check means the figure wasn't available to verify (not a failure).
137         ``ok``
138         requires the per-row balance walk *plus* every available check to pass.
139         """
140         n: int
141         reconciles: bool
142         debit_count_ok: bool | None = None
143         credit_count_ok: bool | None = None
144         opening_ok: bool | None = None
145         closing_ok: bool | None = None
146
147         @property
148         def ok(self) -> bool:
149             checks = [self.debit_count_ok, self.credit_count_ok, self.opening_ok,
150 self.closing_ok]
151             return self.n > 0 and self.reconciles and all(c is True for c in checks)
152
153         @property
154         def confidence(self) -> Confidence:
155             return Confidence.RECONCILED if self.ok else Confidence.UNRECONCILED
156
157     def verify(
158         rows: Sequence[ConceptRow], concepts: StatementConcepts, tol: float = 0.01
159     ) -> BookendReport:
160         """Bookend a parse against the bank's printed SUMMARY: reconciliation + the printed
161         figures.
162
163         The per-row walk catches any mid-statement error/omission; the Dr/Cr counts catch
164         any
165         extra/missing row; the opening & closing balances catch a missing first/last row.
166         Each printed
167         figure is only checked when present (``None`` ? not checked). The opening balance is
168         checked by
169         implication (``balance[0] ? credit[0] + debit[0]`` should equal the printed
170         opening).
171         """
172         reconciles = reconcile_running_balance(_row_entries(rows), tol).ok
173         debit_count_ok = credit_count_ok = opening_ok = closing_ok = None
174         if concepts.debit_count is not None:
175             debit_count_ok = sum(1 for r in rows if (r.debit or 0.0) > 0) ==
176 concepts.debit_count
177         if concepts.credit_count is not None:
178             credit_count_ok = sum(1 for r in rows if (r.credit or 0.0) > 0) ==
179 concepts.credit_count
180         if rows and rows[-1].balance is not None and concepts.closing_balance is not None:
181             closing_ok = abs(rows[-1].balance - concepts.closing_balance) <= tol
182         if rows and rows[0].balance is not None and concepts.opening_balance is not None:
183             implied_opening = rows[0].balance - (rows[0].credit or 0.0) + (rows[0].debit or
184 0.0)
185             opening_ok = abs(implied_opening - concepts.opening_balance) <= tol

```

```

178         return BookendReport(
179             n=len(rows),
180             reconciles=reconciles,
181             debit_count_ok=debit_count_ok,
182             credit_count_ok=credit_count_ok,
183             opening_ok=opening_ok,
184             closing_ok=closing_ok,
185         )
186
187
188     def select_reconciling(candidates: Sequence[Sequence[ConceptRow]], tol: float = 0.01) ->
int | None:
189         """Among candidate concept-row mappings, return the index of the first that
reconciles.
190
191         This is the *selection* half of "extraction proposes, reconciliation disposes": when
the
192         extraction ladder can't be sure which column?concept assignment is right, it
proposes several
193         and lets the per-row walk pick the correct one. Returns ``None`` if none reconcile.
194
195         The candidate *generation* (the L2 mapping search) is Step 2 of the architecture
plan; this is
196         the verifier-side seam it plugs into. Selection here uses the per-row walk, so
candidates need a
197         running balance ? balance-less statements select via :func:`reconcile_total` at the
call site.
198         """
199         for i, rows in enumerate(candidates):
200             if reconcile_running_balance(_row_entries(rows), tol).ok:
201                 return i
202         return None
203
204
205     __all__ = [
206         "parse_amount",
207         "ReconcileReport",
208         "reconcile_running_balance",
209         "reconcile_total",
210         "Confidence",
211         "BookendReport",
212         "verify",
213         "select_reconciling",
214     ]
215

```

43. user (2026-06-10T16:16:25.157Z)

```

src\muneem\parsers\au.py:48:             raise ParseError(f"failed to parse AU statement
{redact_filename(pdf_path)}") from exc
C:\Users\avidu\Projects\muneem\src\muneem\parsers\au.py-49-
src\muneem\parsers\au.py-50-             if not parsed.verified:
C:\Users\avidu\Projects\muneem\src\muneem\parsers\au.py-51-                 dbmod.record_parse(
--
src\muneem\parsers\axis.py:81:             raise ParseError(f"failed to parse Axis statement
{name}") from exc
C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py-82-
src\muneem\parsers\axis.py-83-             if not parsed.verified:
C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py-84-                 dbmod.record_parse(
--
src\muneem\parsers\hdfc.py:434:             raise ParseError(f"failed to parse HDFC statement
{redact_filename(pdf_path)}") from exc
src\muneem\parsers\hdfc.py-435-             for t in stmt.transactions:
C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py-436-
dbmod.insert_hdfc_transaction(

```

44. user (2026-06-10T16:16:33.243Z)

```

60         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
61             if sender and "axisbank.com" in sender.lower():
62                 return True
63             t = text or ""
64             return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement"
in t)
65
66         def parse_and_insert(
67             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
68         ) -> int:
69             """Persist a savings parse **only if it reconciles**; otherwise store nothing.
70
71             Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a
layout the
72             parser can't yet read surfaces loudly instead of saving mislabeled rows.
73             """
74             try:
75                 parsed = parse_axis_savings(pdf_path, password)
76             except ParseError:
77                 raise
78             except Exception as exc:
79                 # Filename only ? never statement text; original cause kept via `from`.
80                 name = redact_filename(pdf_path)
81                 raise ParseError(f"failed to parse Axis statement {name}") from exc
82
83             if not parsed.verified:
84                 dbmod.record_parse(
85                     conn,
86                     doc_id,
87                     confidence=Confidence.UNRECONCILED.value,
88                     parser="AxisParser",
89                     parser_version=__version__,
90                 )
91             return 0
92
93             for r in parsed.rows:
94                 dbmod.insert_axis_transaction(
95                     conn,
96                     document_id=doc_id,
97                     account_number=parsed.account_number,
98                     txn_date=r.txn_date,
99                     value_date=r.value_date or "",
100                     particulars=r.narration,
101                     debit=r.debit,
102                     credit=r.credit,
103                     balance=r.balance,
104                 )

```

45. user (2026-06-10T16:16:34.732Z)

```

src\muneem\config.py:3:muneem is local-first: config lives in the OS config dir, data (DB +
staging)
src\muneem\config.py:23:STAGING_DIR = DATA_DIR / "staging"
src\muneem\config.py:34:     staging_dir: Path = STAGING_DIR
src\muneem\config.py:48:     """Create the local config/data/staging dirs if missing. Never
touches the repo."""
src\muneem\config.py:49:     for directory in (CONFIG_DIR, DATA_DIR, STAGING_DIR):
src\muneem\cli.py:88:     typer.echo(f"staging      : {cfg.staging_dir}")
src\muneem\unlock.py:41:def write_decrypted(path: Path | str, password: str | None, dest: Path |

```

```

str) -> Path:
src\muneem\unlock.py:79:         out = write_decrypted(src, pw, dest) if dest else None
src\muneem\unlock.py:88:         out = write_decrypted(src, entered, dest) if dest else
None

```

46. user (2026-06-10T16:16:37.137Z)

```

Name
  Length
----
-----
April Credit Card Statement.json
2525
lder Credit Card Statement.json
6074
client_secret_207319377548-j0vbfmqak4cq6kqa4s9ikq6l5sgore.apps.googleusercontent.com.json
413

```

47. user (2026-06-10T16:16:52.392Z)

```

array of 12 items
account_number, txn_date, particulars, merchant_category, amount, type

```

48. user (2026-06-10T16:16:53.960Z)

```

top: installed
  installed: client_id, project_id, auth_uri, token_uri, auth_provider_x509_cert_url,
client_secret, redirect_uris

```

49. user (2026-06-10T16:17:10.095Z)

```

1      """Enforce muneem's commercial-license principle (DESIGN.md § 5 rule 8).
2
3      Every declared dependency must be on the permissive allowlist below (kept in sync
4      with docs/DEPENDENCIES.md). Adding a dependency without recording its
5      MIT/BSD/Apache/? license here fails the build, forcing a license review ? and
6      keeps banned copyleft licenses (notably AGPL, e.g. PyMuPDF) out of the project.
7      """
8
9      from __future__ import annotations
10
11     import re
12     import tomllib
13     from pathlib import Path
14
15     REPO_ROOT = Path(__file__).resolve().parents[1]
16
17     # Normalized (lowercased) dependency name -> license. Permissive / commercial-safe only.
18     ALLOWLIST = {
19         "typer": "MIT",
20         "platformdirs": "MIT",
21         "keyring": "MIT",
22         "pdfminer.six": "MIT",
23         "pypdfium2": "Apache-2.0/BSD-3-Clause",
24         "pytesseract": "Apache-2.0",
25         "pillow": "HPND",
26         "pypdf": "BSD-3-Clause",
27         "google-api-python-client": "Apache-2.0",
28         "google-auth": "Apache-2.0",
29         "google-auth-oauthlib": "Apache-2.0",
30         "pdfplumber": "MIT",
31         # SQLCipher at-rest encryption. Binding (sqlcipher3) = Zlib; bundled native deps are
32         # SQLCipher core = BSD-3-Clause, SQLite = Public Domain, OpenSSL = Apache-2.0 ? all
33         # permissive (see docs/DEPENDENCIES.md).

```

```

34     "sqlcipher3": "Zlib",
35     # PyCA cryptography ? Argon2id + AES-GCM for the passphrase-wrapped recovery file.
36     "cryptography": "Apache-2.0 OR BSD-3-Clause",
37     "pytest": "MIT",
38     "pytest-cov": "MIT",
39     "ruff": "MIT",
40     "pyyaml": "MIT",
41     "mypy": "MIT",
42     "msal": "MIT",
43     "requests": "Apache-2.0",
44 }
45
46 BANNED_SUBSTRINGS = ("GPL", "AGPL", "AFFERO")
47
48
49 def _declared_dependency_names() -> set[str]:
50     data = tomlib.loads((REPO_ROOT / "pyproject.toml").read_text(encoding="utf-8"))
51     specs = list(data["project"].get("dependencies", []))
52     for extra in data["project"].get("optional-dependencies", {}).values():
53         specs.extend(extra)
54     names = set()
55     for spec in specs:
56         # "pdfminer.six>=20240706" / "typer[all]>=0.12" -> bare name
57         name = re.split(r"[<=>!~;\[\ ]", spec, maxsplit=1)[0].strip().lower()
58         if name:
59             names.add(name)
60     return names
61
62
63 def test_every_dependency_is_allowlisted():
64     unknown = sorted(_declared_dependency_names() - set(ALLOWLIST))
65     assert not unknown, (
66         f"dependencies not on the commercial-safe allowlist: {unknown}. "
67         "Add them to docs/DEPENDENCIES.md and ALLOWLIST with their license, or remove
68     them."
69 )
70
71 def test_allowlist_contains_no_banned_licenses():
72     bad = {
73         name: lic
74         for name, lic in ALLOWLIST.items()
75         if any(token in lic.upper() for token in BANNED_SUBSTRINGS)
76     }
77     assert not bad, f"copyleft licenses are banned but present in allowlist: {bad}"
78

```

50. user (2026-06-10T16:17:11.661Z)

```

.gitignore:5:testdata/* testdata/credentials/x.json
.gitignore:5:testdata/* testdata/snapshots/x.json
.gitignore:14:.env .env
.gitignore:23:password_rules.yaml password_rules.yaml
.gitignore:29:*.log notes.log
---exit 0

```

51. user (2026-06-10T16:17:27.151Z)

```

5156b5a feat(parsers): Axis savings rewrite onto the concept core (clustering + total reconcile)
(#25)
128fc36 docs(roadmap): flag Axis savings parser as a must-fix (needs thorough rewrite) (#20)
---
---
count: 568
size: 838

```

in-pack: 116
packs: 1
size-pack: 25906

52. user (2026-06-10T16:17:27.157Z)

```
1      # Ground-truth answer key for the promo regression corpus. See docs/TESTING.md sections
2-4.
2      #
3      # `verified: true` -> confirmed by text-layer extraction and/or a visual read; tests
assert it.
4      # `verified: false` -> placeholder to establish during ROADMAP A.4 (OCR + manual
glance); NOT asserted.
5      # `tier`: T1 text-layer code | T2 image-only code (needs OCR) | T3 no code (precision) |
T4 unverified.
6
7      dataquest.pdf:
8          issuer: Dataquest
9          tier: T1
10         code_location: text
11         codes: [JS231sz]
12         expiry: { type: relative, raw: "Ends Today" }
13         verified: true
14
15     makemytrip-2.pdf:
16         issuer: MakeMyTrip
17         tier: T1
18         code_location: text
19         codes: [EARLYBIRDDEAL]
20         expiry: { type: conditional, raw: "at least 15 days in advance" }
21         verified: true
22
23     swiggy.pdf:
24         issuer: Swiggy          # offer brand: ClearTax
25         tier: T2
26         code_location: image
27         codes: [CTSWIGGYUX]
28         expiry: { type: absolute, raw: "before July 31st" }
29         verified: true
30
31     flipkart.pdf:
32         issuer: Flipkart
33         tier: T3
34         code_location: none
35         codes: []
36         expiry: { type: relative, raw: "20-25 Jul" }
37         verified: true
38
39     airasia-1.pdf:
40         issuer: Air India Express # NB: filename says "airasia" but the mailer is Air India
Express
41         tier: T4
42         code_location: unknown
43         codes: []
44         verified: false
45
46     airasia-2.pdf:
47         issuer: AirAsia
48         tier: T4
49         code_location: unknown
50         codes: []
51         verified: false
52
53     airasia-3.pdf:
54         issuer: AirAsia
```

```

55     tier: T4
56     code_location: unknown
57     codes: []
58     verified: false
59
60     makemytrip-1.pdf:
61         issuer: MakeMyTrip
62         tier: T4
63         code_location: unknown
64         codes: []
65         verified: false
66
67     unacademy.pdf:
68         issuer: Unacademy
69         tier: T4
70         code_location: unknown
71         codes: []
72         verified: false
73
74     tataneu.pdf:
75         issuer: Tata Neu
76         tier: T4
77         code_location: unknown
78         codes: []
79         verified: false
80

```

53. user (2026-06-10T16:17:35.470Z)

```

1     """Authorize IMAP (read-only) and list the statement emails found.
2
3     Run by *you* ? it prompts for an App Password, caches it in your OS keychain,
4     then prints the statement emails it can see.
5
6     python tools/imap_auth.py --host imap.mail.yahoo.com --user user@yahoo.com
7     """
8
9     from __future__ import annotations
10
11     import argparse
12
13
14     def main() -> int:
15         parser = argparse.ArgumentParser(description="Authorize IMAP and list statement
emails.")
16         parser.add_argument("--host", required=True, help="IMAP server hostname.")
17         parser.add_argument("--user", required=True, help="IMAP username.")
18         parser.add_argument("--sender", default="hdfcbank.net", help="Sender email domain to
search.")
19         parser.add_argument("--max", type=int, default=25, help="Max emails to list.")
20         args = parser.parse_args()
21
22         from muneem.ingestion.base import SearchCriteria
23         from muneem.ingestion.imap import IMAPProvider
24
25         print(f"Host : {args.host}")
26         print(f>User : {args.user}")
27
28         provider = IMAPProvider(args.host, args.user)
29         provider.authorize()
30
31         criteria = SearchCriteria(sender=args.sender, has_attachment=True,
newer_than_days=None)
32
33         ids = provider.search_messages(criteria, max_results=args.max)

```

```

34         print(f"Found {len(ids)} message(s) matching criteria:\n")
35         for mid in ids:
36             hdrs = provider.get_headers(mid)
37             attachments = ", ".join(name for name, _ in provider.list_pdf_attachments(mid))
38             or "(none)"
39             date = hdrs.get("date", "")[:16]
40             subject = hdrs.get("subject", "(no subject)")[:55]
41             print(f" {date:16} {subject:55} {attachments}")
42         print("\nDone.")
43         return 0
44
45     if __name__ == "__main__":
46         raise SystemExit(main())
47

```

54. user (2026-06-10T16:17:36.406Z)

No matches found

55. user (2026-06-10T16:17:45.958Z)

```

1     """IMAP email provider ? ?? EXPERIMENTAL / INCOMPLETE.
2
3     Basic auth + search are implemented; ``get_headers`` and the attachment methods are
4     ``NotImplementedError``. This provider is not wired into any CLI command and is not
5     on a supported path yet (see docs/CURRENT_STATE.md §1). Do not rely on it; it is excluded
6     from the strict mypy gate (pyproject ``[[tool.mypy.overrides]]``) until the ``fetch`` command
7     lands (ROADMAP B.4 / feedbackJune2026 P7).
8     """
9
10    from __future__ import annotations
11
12    import imaplib
13    from pathlib import Path
14
15    from muneem import secrets
16
17    from .base import EmailProvider, SearchCriteria
18
19    _PASS_KEY = "imap_password"
20
21
22    class IMAPProvider(EmailProvider):
23        def __init__(self, host: str, username: str):
24            self.host = host
25            self.username = username
26            self.mail = None
27
28        def authorize(self) -> None:
29            password = secrets.get_secret(f"{_PASS_KEY}_{self.username}")
30            if not password:
31                import getpass
32
33                password = getpass.getpass(f"Enter App Password for {self.username} at {self.host}: ")
34                secrets.set_secret(f"{_PASS_KEY}_{self.username}", password)
35
36            self.mail = imaplib.IMAP4_SSL(self.host)
37            self.mail.login(self.username, password)
38            self.mail.select("inbox", readonly=True)
39

```

```

40     def _build_search(self, criteria: SearchCriteria) -> list[str]:
41         search_args = []
42         if criteria.sender:
43             search_args.extend(["FROM", f'"{criteria.sender}"'])
44         # IMAP doesn't reliably support native "has attachment" searching natively on
all servers
45         # We can fetch structure later to filter, but we omit it from standard search
46         return search_args
47
48     def search_messages(self, criteria: SearchCriteria, max_results: int = 100) ->
list[str]:
49         if not self.mail:
50             self.authorize()
51
52         search_args = self._build_search(criteria)
53         if not search_args:
54             search_args = ["ALL"]
55
56         status, messages = self.mail.search(None, *search_args)
57         if status != "OK":
58             raise RuntimeError("IMAP search failed")
59
60         msg_ids = messages[0].split()
61         return [mid.decode() for mid in msg_ids[-max_results:]]
62
63     def get_headers(self, message_id: str) -> dict[str, str]:
64         raise NotImplementedError(
65             "IMAPProvider.get_headers is not yet implemented. "
66             "Attachment download and header normalization are pending."
67         )
68
69     def list_pdf_attachments(self, message_id: str) -> list[tuple[str, str]]:
70         raise NotImplementedError(
71             "IMAPProvider.list_pdf_attachments is not yet implemented. "
72             "Attachment download and header normalization are pending."
73         )
74
75     def download_attachment(self, message_id: str, attachment_id: str, dest_dir: Path)
-> Path:
76         raise NotImplementedError(
77             "IMAPProvider.download_attachment is not yet implemented. "
78             "Attachment download and header normalization are pending."
79         )
80

```

56. user (2026-06-10T16:18:04.362Z)

```

1     """Tests for the pre-commit sensitive-file guard (.github/precommit_check.py).
2
3     Tests the pure rule function + the exit-code path (via PRECOMMIT_TEST_FILES), so
4     no shell, git mutation, or real staging is needed ? portable across OSes.
5     """
6
7     from __future__ import annotations
8
9     import importlib.util
10    from pathlib import Path
11
12    import pytest
13
14    REPO_ROOT = Path(__file__).resolve().parents[1]
15    CHECK = REPO_ROOT / ".github" / "precommit_check.py"
16
17
18    def _load_checker():

```

```

19         spec = importlib.util.spec_from_file_location("precommit_check", CHECK)
20         assert spec and spec.loader
21         mod = importlib.util.module_from_spec(spec)
22         spec.loader.exec_module(mod)
23         return mod
24
25
26     pc = _load_checker()
27
28
29     @pytest.mark.parametrize(
30         "path",
31         [
32             "testdata/hdfc-statements/Avi_Dullu_28022026.pdf",
33             "testdata/axis-statements/AXIS BANK Statement for October 2025.pdf", # spaces
34             "testdata/aubank-statements/logs_71559053525.zip", # non-PDF under testdata/
35             "testdata/credentials/client_secret.json",
36             "some/dir/Statement.PDF", # PDF anywhere, any case
37             "muneem.db",
38             "data/ledger.sqlite3",
39             "data/muneem.db.plaintext.bak", # cleartext backup from the encrypt migration
40             "muneem-recovery.json", # passphrase-wrapped DB-key recovery file
41             "backups/my-recovery.json",
42             "credentials.json",
43             "src/token.json",
44             "oauth_token_cache",
45             "password_rules.yaml",
46             "password_rules.yml",
47             "password_rules.json",
48             ".env",
49             ".env.local",
50             "secrets/server.pem",
51             "id_rsa.key",
52         ],
53     )
54     def test_sensitive_paths_are_blocked(path):
55         assert pc.is_sensitive(path) is not None, f"should block: {path}"
56
57
58     @pytest.mark.parametrize(
59         "path",
60         [

```

57. user (2026-06-10T16:18:05.239Z)

```

src\muneem\db.py:50:    source_text TEXT
src\muneem\cli.py:114:    parser = get_parser(ex.text)
src\muneem\cli.py:164:    parser = get_parser(ex.text)
src\muneem\cli.py:252:    text_fields = extract_offer(ex.text)
src\muneem\cli.py:295:    source_text=ex.text[:2000],

```

58. user (2026-06-10T16:18:41.484Z)

```

38-| requests | Apache-2.0 | HTTP library for Outlook provider |
39-| pdfplumber | MIT | text/table + coordinate extraction for statements (HDFC/Axis/AU) |
40-| sqlalchemy | MIT | at-rest DB encryption (SQLCipher, transparent AES-256) ? see below |
41-| cryptography | Apache-2.0 / BSD-3 | Argon2id + AES-GCM for the passphrase-wrapped DB-key
recovery file (PyCA) |
42-| pytest *(dev)* | MIT | tests |
--
57-## Current / recently added (permissive)
58-
59- **sqlcipher3** (Zlib) ? at-rest encryption of the local ledger (SQLCipher, transparent
60- AES-256). Its prebuilt wheels **bundle** native code whose licenses are all permissive:
61- **SQLCipher core = BSD-3-Clause, SQLite amalgamation = Public Domain, OpenSSL =

```

Apache-2.0**.

59. user (2026-06-10T16:18:42.485Z)

```
src\muneeem\ocr.py:36: cmd = os.environ.get("MUNEEM_TESSERACT_CMD")
```

60. user (2026-06-10T16:21:08.503Z)

Structured output provided successfully

61. assistant (2026-06-10T16:21:13.668Z)

Audit complete. Findings returned via structured output.